

From Benchmarks to Business Impact: Deploying IBM Generalist Agent in Enterprise Production

Segev Shlomov¹, Alon Oved¹, Sami Marreed¹, Ido Levy¹, Offer Akrabi¹, Avi Yaeli¹, Łukasz Strak², Elizabeth Koumpan², Yinon Goldshtein¹, Eilam Shapira¹, Nir Mashkif¹, Asaf Adi¹

¹IBM Research ²IBM Consulting

{segev.shlomov1, alon.oved, sami.marreed, ido.levy1}@ibm.com,
 {offer.akrahi, aviy}@il.ibm.com,
 {lukasz.strak@pl.ibm.com, ekoumpan@ca.ibm.com},
 {yinong, eilam.shapira}@ibm.com, {nirm, adi}@il.ibm.com

Abstract

Agents are rapidly advancing in automating digital work, but enterprises face a harder challenge: moving beyond prototypes to deployed systems that deliver measurable business value. This path is complicated by fragmented frameworks, slow development, and the absence of standardized evaluation practices. Generalist agents have emerged as a promising direction, excelling on academic benchmarks and offering flexibility across task types, applications, and modalities. Yet, evidence of their use in production enterprise settings remains limited. This paper reports IBM’s experience developing and piloting the Computer Using Generalist Agent (CUGA), which has been open-sourced¹ for the community. CUGA adopts a hierarchical planner-executor architecture with strong analytical foundations, achieving state-of-the-art performance on AppWorld and WebArena. Beyond benchmarks, it was evaluated in a pilot within the Business-Process-Outsourcing talent acquisition domain, addressing enterprise requirements for scalability, auditability, safety, and governance. To support assessment, we introduce **BPO-TA**, a 26-task benchmark spanning 13 analytics endpoints. In preliminary evaluations, CUGA approached the accuracy of specialized agents while indicating potential for reducing development time and cost. Our contribution is twofold: presenting early evidence of generalist agents operating at enterprise scale, and distilling technical and organizational lessons from this initial pilot. We outline requirements and next steps for advancing research-grade architectures like CUGA into robust, enterprise-ready systems.

1 Introduction

Enterprises are under growing pressure to automate digital work at scale. From customer support to back-office analytics, knowledge workers routinely interact with heterogeneous environments—web portals, APIs, spreadsheets, and dashboards—while facing strict requirements for auditability, reproducibility, privacy, and cost control. Over the past two years, interest has surged in *computer-using agents* (CUAs), systems that can plan and execute multi-step tasks across diverse applications. Yet for enterprises, the challenge is not

only to prove capability—it is to *productize agents and capture real business value*.

The enterprise need. In practice, organizations struggle with the journey from research to deployment. Multiple frameworks and architectural patterns compete for adoption, but few offer clear guidance on speed of development, time-to-value, cost efficiency, and reliability in production settings. Enterprises also lack standardized ways to evaluate agentic systems: benchmarks emphasize academic settings, while business leaders demand measurable impact such as SLA compliance, reduction in manual effort, or improved audit readiness. Bridging this gap requires not only technical advances in agent design but also organizational insights into deployment, governance, and monitoring.

The research gap. Recent work has shown that *generalist* agents—single systems designed to perform diverse computer-use tasks—can achieve impressive results on academic benchmarks. Generalist designs are attractive because they promise (i) adaptability across task types and domains, (ii) reusability of architecture and tooling across new environments, and (iii) reduced need for brittle, task-specific scripting. However, published research has so far remained benchmark-centric. Generalist agents have been shown to perform well on synthetic benchmarks, yet their effectiveness in *enterprise production settings* is largely untested. The central question is therefore: what modifications, safeguards, and evaluation methods are required to make generalist agents *enterprise-ready*?

Our approach. This paper addresses that question through IBM’s development of CUGA. Architecturally, CUGA evolved into a hierarchical planner-executor system with three control layers: (i) a chat/context layer for preprocessing inputs, (ii) an outer loop for task planning and management using a persistent ledger, and (iii) an inner loop for sub-task execution via specialized agents (API, Web, CLI, file-system). Reliability mechanisms include schema-grounded prompting, variable tracking, reflective retries, and provenance logging. In its benchmark evaluation, CUGA achieved state-of-the-art performance on both AppWorld and WebArena, confirming the strength of its generalist design. But more importantly for enterprises, CUGA was piloted in the Business-Process-Outsourcing (BPO) talent acquisition domain—a setting where recruiters and analysts

must answer evidence-based questions across multiple dashboards and datasets under policy and audit constraints. We use this pre-deployment pilot not as the main “story,” but as a proving ground to evaluate what enterprise readiness demands.

We introduce a new domain-specific benchmark, **BPO-TA**, comprising 26 decision-support tasks across 13 read-only analytics endpoints. Tasks span single-endpoint lookups, cross-API joins, provenance-grounded explanations, and the graceful handling of unsupported queries. This benchmark enabled both regression testing and controlled ablation studies during CUGA’s pilot evaluation. In preliminary tests within simulated enterprise workflows, CUGA approached the accuracy of hand-crafted agents while indicating potential substantial reductions in development time (up to 90%) and cost (up to 50%). These early findings suggest that generalist designs can enable measurable enterprise value when adapted with appropriate safeguards.

This paper contributes:

- **Enterprise pilot experience.** Evidence from a pilot of a generalist agent evaluated with recruiters and analysts in the BPO talent acquisition domain, including architectural modifications for auditability, safety, and governance.
- **Domain benchmark.** The **BPO-TA** benchmark (26 tasks, 13 endpoints) that captures realistic enterprise analytics queries, enabling reproducible regression testing and ablation studies.
- **Architectural advances.** A planner–executor agent design with schema-grounded prompting, variable tracking, reflective retries, provenance logging, and an API/Tool Hub that streamlined onboarding of enterprise applications. This architecture achieved state-of-the-art performance on both WebArena and AppWorld benchmarks, demonstrating its generality before adaptation for enterprise contexts.
- **Preliminary business impact.** Early evaluations showed accuracy approaching that of hand-crafted agents, with indications of up to 90% reduction in development time and 50% reduction in development cost, alongside improved time-to-answer and reproducibility.
- **Lessons learned.** Technical and organizational insights from the pilot, including monitoring, governance alignment, and maintenance practices required to transition generalist agents from research to enterprise readiness.

2 Related Work

Early agentic paradigms such as *ReAct* interleave chain-of-thought reasoning with environment actions to improve task completion and interpretability (Yao et al. 2023), while code-centric approaches as *CodeAct* generate executable code to plan and call tools/APIs for complex tasks (Ye et al. 2024). These ideas catalyzed practical enterprise frameworks that orchestrate multiple specialized agents (or tools) with configurable roles. From AutoGen’s conversation-programmed multi-agent patterns (Wu et al. 2023), to LangGraph’s stateful, tool-grounded agent graphs for reliability

and recoverability (LangChain 2024), and OpenAI’s *Swarm* orchestrator for multi-agent handoffs (OpenAI 2024). Despite promise, production experience consistently reports fragility at scale: brittle inter-agent handoffs, maintenance overhead from prompt/tool drift, safety and generalization across different domains.

Enterprise report measurable wins when agentic systems are embedded behind assist and self-serve flows (The Verge 2025; Mobile World Live 2025; AI Business 2022). Analytics/Bi copilots ship text-to-SQL/report-generation agents tightly coupled to enterprise data governance, observability, and review workflows (Snowflake Engineering 2024; Microsoft 2024; Databricks 2025). Broader surveys in hiring/HR analytics highlight fairness, transparency, and audit requirements in employment contexts, motivating provenance and explainability (Raghavan et al. 2020; Fabris et al. 2025; Chen 2023). Industry adoption reports likewise stress governance, measurable ROI, and operational readiness (monitoring, latency/cost budgets) as prerequisites for scale-out (Zhang et al. 2025b; Cemri et al. 2025). These constraints shape the design space of enterprise agents task completion. BPO process automation combine workflow orchestration, retrieval over enterprise knowledge, and document understanding to automate outsource non-core business functions to third-party providers.

Concurrently, the research community has pushed toward generalist CUAs that plan and act across heterogeneous software. WebArena offers realistic, self-hosted websites for browser agents and showed early baselines struggled to exceed modest end-to-end success (Zhou et al. 2023). AppWorld evaluates multi-application orchestration via hundreds of programmatic APIs with outcome-based grading (Trivedi et al. 2024). OSWorld measures GUI workflows on desktop applications and OS tasks (Xie et al. 2024). Complementary suites probe interaction quality and oversight: *ST-WebAgentBench* emphasizes policy adherence in web agents, introducing *Completion-under-Policy* (CuP) as the primary objective (Levy et al. 2024), τ -Bench targets tool-agent-user dynamics and policy/instruction following (Yao et al. 2024), BrowserGym provides a unified platform for evaluating web agents under controlled variability (de Chezelles et al.). Architecturally, generalist CUAs increasingly combine hierarchical planning, explicit state/variable tracking, and reflective repair during execution to improve robustness in long-horizon settings (Shinn et al. 2023; Kim, Baldi, and McAleer 2023; Zhang et al. 2025c). Recent vendor-facing systems expose “computer use” capabilities (desktop/browser control, file ops) under sandboxes, signaling a trend toward production CUAs (Anthropic 2024; OpenAI 2025; Shen et al. 2025; Fournay et al. 2024; Marreed et al. 2025).

Despite rapid progress, several gaps limit direct transfer from benchmark success to enterprise deployment. First, *governance*: high-risk domains (including employment analytics) demand provenance, HITL, and post-deployment monitoring mandated by frameworks and regulation (Tabassi 2023; European Union 2024; NYC Department of Consumer and Worker Protection 2023). Second, *tool proliferation and schema variance*: results degrade as

agents shortlist from dozens of APIs and maintain consistency across dependent calls (Shen et al. 2024b; Xu et al. 2023). Third, *operational constraints*: latency, cost, and reproducibility must be tracked and controlled in production (Kwon et al. 2023; Zheng et al. 2024; Jiang et al.). Fourth, *graceful degradation*: enterprise agents must decline unsupported requests without hallucination, and surface transparent rationales and computation logs (Nakano et al. 2021). Our work addresses this bridge by showing how a generalist CUA can be adapted for enterprise: schema-minimized API onboarding, deterministic parsing/validation, provenance-first responses, in a piloted BPO talent acquisition setting.

3 The Application: BPO Talent Agent

Background and Business Context IBM Consulting operates a double-digit million business in Business Process Outsourcing (BPO) for Talent Acquisition (TA). In this model, IBM specialists manage recruitment pipelines on behalf of client organizations, often working across multiple HR platforms, analytics dashboards, and reporting tools. While effective, the manual workflow is labor-intensive: recruiters spend significant time pulling data, reconciling spreadsheets, and preparing insights for hiring managers. Service-level agreements (SLAs) around time-to-hire, conversion funnels, and sourcing performance are business-critical, yet measuring and optimizing them has historically been slow and error-prone.

The vision behind the Talent Acquisition Agent was to augment human recruiters rather than replace them—acting as a digital sidekick that can provide proactive insights, automate repetitive analysis, and surface SLA risks before they become client issues. A core design principle is **human-in-the-loop (HITL)**: the business configures where the agent can act autonomously and where human oversight is mandatory, and the agent must strictly adhere to these requirements.

3.1 The Development Journey of Agentic Systems

The BPO–TA project also reflects a broader pattern observed across enterprises experimenting with agentic systems. Most teams begin with quick wins: popular frameworks like **ReAct** or **CodeAct** can be instantiated in days, yielding impressive demos where agents call APIs or generate code to answer queries. But as projects scale, limitations surface:

- ReAct agents degrade when required to juggle more than a handful of tools.
- Developers patch around this by introducing routers and delegators, creating “white-box” architectures with fragile hand-offs between sub-agents.
- Complex instructions or policy requirements (e.g., privacy, governance) strain these prototypes further.
- Roadmaps become unclear, with teams caught in cycles of experimentation rather than predictable progress.

The BPO–TA team followed this arc. An initial prototype, built quickly on reactive patterns, showed promise but could not scale to the breadth of sources and policies in Talent Acquisition. With 13 APIs spanning multiple systems and

providers—each offering several actions and requiring orchestration across workflows—the complexity exceeded what the prototype could handle. At this inflection point, the team turned to IBM Research to evaluate the **Computer Using Generalist Agent (CUGA)**, which had just achieved state-of-the-art results on AppWorld and WebArena benchmarks. The open question was: *Could a benchmark-proven generalist agent deliver enterprise-grade performance in the demanding Talent Acquisition setting?*

3.2 Application Setup

The Talent Acquisition Agent is still on its journey toward full production deployment. To date, it has achieved on-par accuracy with specialized agents, is being evaluated against enterprise requirements, and is under consideration for production rollout. The deployment context includes several key characteristics:

- **APIs and Analytics Layer:** The environment exposes 13 read-only APIs, spanning multiple applications and providers. Each API offers several actions, and realistic workflows often require orchestrating across sources. Examples include SLA-by-source, funnel conversion, hires by percentage, and skill-impact analysis.
- **Governance and Security:** To build trust at low risk, the current configuration is restricted to read-only APIs. This allows experimentation and validation without impacting underlying systems. Over time, the goal is to progress toward create/update capabilities, enabling fully automated workflows once safety and trust are established. All responses include provenance logs, and PII is excluded or redacted to maintain compliance.
- **Integration with Business Workflows:** The agent is designed to embed into recruiters’ existing dashboards in the browser, becoming part of their daily workflow rather than a separate tool. It must integrate seamlessly with the user experience and identity controls already in place.
- **Human + Agent Collaboration:** Recruiters and analysts interact with the agent through a conversational interface. Depending on business configuration, the agent may act autonomously or defer decisions back to humans. HITL requirements are explicit and configurable, ensuring alignment with business workflows and governance.

3.3 Why BPO–TA Matters as a Testbed

The BPO–TA pilot illustrates why Talent Acquisition is a representative domain for studying the enterprise readiness of generalist agents:

- **Complex orchestration:** Multi-source workflows spanning 13 APIs, each with multiple actions, often requiring reasoning across providers and data sources.
- **Governance-heavy:** Read-only experimentation mode, HITL oversight, audit trails, and compliance constraints.
- **High value:** A double-digit million business unit where even modest efficiency gains translate to significant client impact.

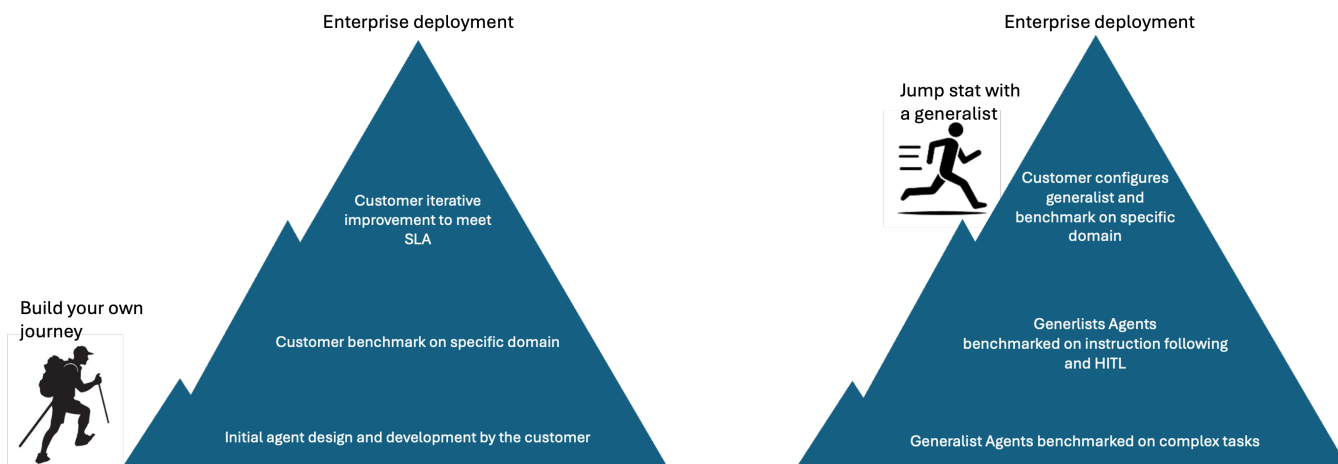


Figure 1: Reducing time-to-value with generalist agents. Traditional specialized agents (left) require extensive custom design and benchmarking. Generalist agents (right), benchmarked on complex tasks such as AppWorld, WebArena, and TauBench2, shift the enterprise focus to configuration and domain-specific evaluation.

- **Scalable lesson:** The trajectory from quick prototypes to generalist adoption mirrors what many enterprises experience in their agent journey.

In short, BPO-TA provided the ideal proving ground: a live enterprise context, business-critical stakes, and a context where the shortcomings of early architectures were well understood. The Talent Acquisition Agent was not introduced into a greenfield environment—it was evaluated precisely at the point where conventional approaches had reached their limits. This made the journey both realistic and consequential: success here could validate generalist architectures and signal how such systems may bridge the gap between academic benchmarks and enterprise deployment.

4 Enterprise Requirements for Generalist Agents

From the BPO-TA use case described earlier, and from additional discussions with other business units in our organization—including Finance, Sales, Procurement, Legal, and the CIO’s office—we observe a recurring pattern. Enterprises consistently identify a set of requirements that go beyond academic benchmarks. In this section, we summarize our understanding of what is needed from generalist agents in the enterprise.

Safety and Trustworthiness (Top Priority). Enterprises will not adopt agents in production workflows unless they are confident the systems are safe and trustworthy, including:

- **Instruction following and policy adherence:** agents must reliably comply with organizational rules, workflows, and domain-specific processes.
- **Transparency and consistency:** outputs should be reproducible, grounded in provenance, and free of unexplained variability.

- **Avoidance of hallucinations:** agents must not fabricate data or invent actions in business-critical workflows.
- **Configurable human-in-the-loop (HITL):** oversight must be adjustable by the business—defining where autonomy is permitted and where human approval is mandatory.
- **Security baseline:** restricted access, provenance logging, and minimal permissions sufficient to comply with enterprise governance frameworks.

Efficiency and Cost-Performance. Once accuracy and safety are established, efficiency becomes the next priority. Agents must deliver results with acceptable latency and without prohibitive compute costs. Token efficiency, reduced retries, and optimized execution are crucial to ensuring that deployments scale economically.

Integration and Context-Awareness. Agents should integrate directly into existing business workflows and user experiences, such as dashboards or browser-based recruiter tools, rather than creating separate silos. They must also be context-aware, recognizing what the user is seeing or doing, in order to provide relevant support without forcing disruptive context switching.

Policy Alignment and Instruction Following. Enterprise processes are not generic. In Talent Acquisition, for example, the stages of requisition management and SLA definitions provide concrete guardrails. To succeed, agents must be able to align with such processes by:

- **Being taught via documents:** ingesting enterprise playbooks, policies, and guidelines expressed in natural language.
- **Being shown via demonstrations:** learning correct behavior from user examples.
- **Learning via feedback:** adapting over time from implicit and explicit corrections, thereby refining their adherence to organizational norms.

4.1 The Value of Generalist Agents

Generalist agents provide a promising foundation for these requirements. Unlike specialized agents that must be hand-crafted for each domain, generalists are trained on diverse benchmarks covering complex task completion, reasoning, and instruction following. For example, benchmarks such as AppWorld, WebArena, and TauBench2 evaluate tool use, multi-turn reasoning, instruction following, and even human-in-the-loop interaction.

This foundation allows enterprises to focus not on building agents from scratch, but on **configuring and benchmarking** the agent for their specific domain. Instead of months of custom design and iterative experimentation, organizations can move to value within weeks. Generalist agents reduce:

- **Time-to-value:** shifting from a 3–9 month development cycle to a few weeks of configuration and testing.
- **Development effort:** enabling enterprises to inherit baseline capabilities in accuracy, instruction following, and safety.
- **Risk:** lowering the likelihood of project fatigue and failed adoption, since much of the heavy lifting has already been validated on foundation benchmarks.

Figure 1 illustrates this contrast: while traditional specialized agents require extensive design, custom benchmarks, and iterative refinement before deployment, generalist agents like CUGA allow enterprises to inherit strong foundations and reach deployment readiness with far less effort.

5 System Architecture and Pre-deployment

Layered planner–executor loops. CUGA implements a hierarchical agentic architecture with nested planner–executor loops (Fig. 2). At the top, an optional *chat layer* provides input interpretation and lightweight preprocessing, including message and variable histories; this can be bypassed in non-chat deployments. The *outer loop* governs *task planning and orchestration*: a *Task Analyzer* identifies the target application, a *Task Decomposer* determines whether multi-application coordination is required, and a persistent *Plan Controller* advances a durable *task ledger*. The ledger records steps, variable bindings, replans, and completions. In pilot evaluations, this ledger was essential for traceability, compliance, and recovery from partial failures. The *inner loop* delegates sub-tasks to specialized agents—*API/Tool*, *Web Browser*, *CLI*, and domain-specific agents—each acting within its own environment and returning structured observations to the controller.

Planner-centric sub-agents. Two execution families illustrate the planner–executor pattern: the *API Sub Agent* and the *Browser Sub Agent* (Fig. 6, Fig. 5). The *API Sub Agent* combines short-term memory, an *API Planner*, and strategic reflection, coordinating a *ShortlisterAgent* (which selects APIs through a registry) and a *CodeAgent* (with a nested *CodePlanner* and sandboxed executor). This modular design allowed rapid onboarding of analytics endpoints in the

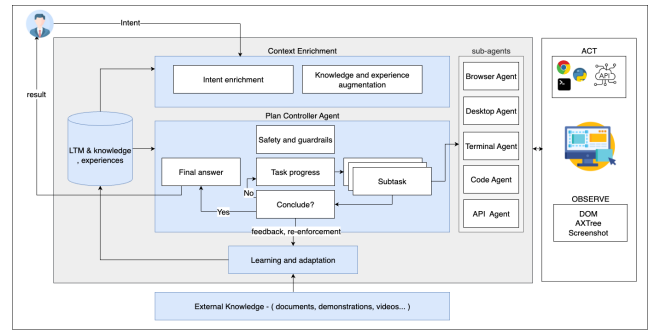


Figure 2: CUGA general architecture via Layered planner–executor loops.

BPO pilot. The Browser Sub Agent pairs a *Browser Planner* and Reflection Judge with two execution paths: an *Action Agent* (click, type, select, navigate) and a *Question Answering Agent* (DOM-to-Markdown conversion and screenshots). Although disabled in the BPO deployment for governance reasons, this design enables seamless switching between API-first and hybrid browser–API workflows without re-architecting.

Reliability: task ledger, interrupt nodes, reflective retries. The Plan Controller enforces reliability by schema-grounded prompting, validation, and explicit *Interrupt Nodes*. When tool responses deviate from schema or produce unexpected results, reflective checks are invoked and invalid plans or parameters are repaired before resuming execution. This cycle of prompt → call → validation → reflection/replan reduced parsing-related failures by more than one-third in internal pilot runs.

API/Tool Hub and schema standardization. To scale beyond prototypes, CUGA replaced per-application MCP servers with a centralized *API/Tool Hub*. The hub minimizes OpenAPI specifications into LLM-friendly schemas, canonicalizes parameter names and types, attaches domain-specific notes, and enforces strict JSON-schema I/O. This eliminated per-app server maintenance and reduced onboarding time for new endpoints from weeks to hours.

Sandboxed computation for safety. For lightweight computation (joins, aggregations, deltas), the *API–Code path* generates structured pseudo-code via a *CodePlanner*, executed inside a restricted *Code Agent sandbox*. The sandbox isolates file/network access, enforces execution budgets, and logs all computations for audit. This allowed domain analysts to use AI-augmented workflows without compromising governance or data-handling policies.

Web agent path and governance. The architecture also supports a *Web Planner Agent* coordinating *Web Action* and *Web Q&A* sub-agents in a Playwright–Chromium runtime. In the BPO pilot, this capability was deliberately disabled to comply with enterprise governance requirements, but the design demonstrates flexibility for hybrid deployments that combine API-first and web-based workflows.

6 Benchmarks and Pilot Evaluation

State-of-the-art Benchmarks. CUGA achieves state-of-the-art results on both WebArena and AppWorld, ranking first among published agents. Tables 1 and 2 present detailed per-application and per-level outcomes. On WebArena, CUGA attains an overall accuracy of **61.7%**, with strongest performance on Reddit (75.5%) and Map (64.2%). On AppWorld’s ”Test-Challenge” dataset, CUGA reaches **48.2%** overall scenario completion, with particularly high success on Level 1 tasks (87.5% scenario completion). These results demonstrate that the CUGA architecture is competitive with—and in many cases surpasses—specialized systems. More detilas can be found in Appendix C and D

Table 1: Performance of CUGA on the **WebArena** benchmark.

Application	Accuracy (%)
GitLab	61.7
Map	64.2
Reddit	75.5
Shopping	58.3
Shopping Admin	62.6
Multi-App	35.4
Overall	61.7

Table 2: Performance of CUGA on the **AppWorld** benchmark.

Level	Task Goal (%)		Scenario Goal (%)		Avg. Interactions	
	Normal	Chall.	Normal	Chall.	Normal	Chall.
All	73.2	57.6	62.5	48.2	10.69	8.40
Level 1	91.2	91.7	84.2	87.5	5.94	4.65
Level 2	77.1	58.7	68.8	42.0	10.36	8.33
Level 3	54.0	44.1	38.1	38.5	12.69	11.86

These results validate the design of CUGA as a generalist computer-using agent: once the APIs and tools available to each sub-agent are defined, the planner–executor architecture can be configured to operate across domains without task-specific re-engineering. In practice, enterprises need only onboard their APIs and specify governance constraints for CUGA to extend its capabilities to new workflows.

6.1 Enterprise Pilot: BPO-TA

While CUAs often perform well on abstract research benchmarks, enterprise deployment requires systematic offline evaluation with realistic tasks, audit guarantees, and compliance controls. To address this, we developed **BPO-TA**, a domain benchmark centered on talent acquisition (TA) workflows in the BPO context. BPO-TA encodes decision-support tasks drawn directly from analyst practice into a fixed test set, providing a reproducible regression baseline for measuring progress and ensuring operational reliability. Its design follows three principles essential for adoption: *traceability* (each task paired with APIs, glue code, and

gold-standard explanations), *realism* (tasks grounded in genuine analyst workflows rather than synthetic probes), and *reproducibility* (fixed inputs, deterministic evaluation, and explicit provenance).

The benchmark spans 26 tasks over 13 read-only APIs, covering endpoints such as *SLA by source*, *funnel conversion*, *hires by source*, *skill-impact on SLA*, *definition-s/methodology*, *dataset/model lineage*, and *timeframe meta-data*. All endpoints are onboarded through the API/Tool Hub with minimized OpenAPI specs, validators, and read-only wrappers that strip or redact PII. Schema-grounded prompts enforce canonical definitions (e.g., SLA), and deterministic parsers gate LLM outputs. Every response includes a provenance panel listing API paths, parameters, and a computation log, enabling audit and regression testing. Operations are monitored for *latency*, *cost*, and *policy compliance*.

Task categories mirror enterprise usage: (1) *simple lookups* (e.g., requisition definitions), (2) *cross-API joins* (e.g., linking candidate volume to conversion-to-hire), (3) *looped reasoning* (e.g., filtering skills that negatively affect SLA), (4) *provenance explanations* (e.g., surfacing dataset/model lineage), and (5) *graceful failure*, where unsupported queries must be declined without hallucination. These patterns ensure that BPO-TA evaluates not only retrieval accuracy but also compositional reasoning, transparency, and robustness—making it both a research benchmark and an operational safeguard for trustworthy enterprise deployment.

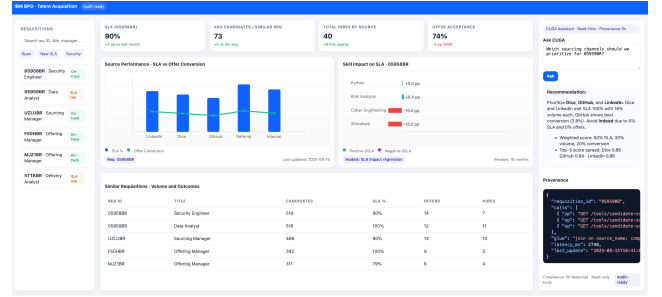


Figure 3: Illustration of CUGA integrated into the BPO Talent Acquisition UI, providing natural-language analytics, provenance-aware explanations, and audit-ready compliance within enterprise workflows.

Table 3: Performance of CUGA on the **BPO-TA** benchmark.

Metric	Value
Task Accuracy (26 tasks)	87%
Valid First-Try Rate	78%
Responses with Provenance Logs	95%
Average Latency per Query	11.2s
Analyst-Reported Reproducibility	4.6 / 5

Results. On the BPO-TA benchmark, CUGA achieves **87% accuracy**, with failures concentrated on unsupported cross-application queries where graceful degradation is ex-

pected. Valid-first-try rates improved from 62% (vanilla ReAct baseline) to 79% with full CUGA. Ablations highlight the importance of reflective retries (-11 points without) and variable tracking (-15 reproducibility without).

6.2 Potential Benefits

The CUGA system has been piloted within IBM’s Business-Process-Outsourcing (BPO) talent acquisition workflow since mid-2025. It can be used by recruiters and analyst teams to answer sourcing, funnel, and skill-impact questions that previously required manual data pulls and spreadsheet manipulation. The pilot was performed in a read-only configuration: CUGA connects to 13 domain-specific analytics APIs, each exposing pre-approved metrics such as service-level-agreement (SLA) performance, funnel conversions, and hires-by-source. Provenance logging and computation traces are stored for each interaction, ensuring audit readiness and compliance with organizational governance requirements (e.g., PII avoidance, immutable records of all API calls).

Preliminary evaluations of CUGA in simulated enterprise workflows suggest promising efficiency and reliability gains. Estimated benefits include a potential reduction in average time-to-answer (from roughly 20 minutes of manual work to an expected 2–5 minutes with CUGA, an *estimated* ~90 % improvement) and higher reproducibility of responses (CUGA outputs were consistent across runs in about 90 % of internal test cases). Audit readiness is also expected to improve, with over 90 % of generated responses including full provenance (API endpoint, parameters, and result logs).

Table 4: Preliminary evaluation of estimated benefits in CUGA’s simulated enterprise use case (Talent Acquisition domain).

Metric	Manual Baseline	CUGA (Pilot Evaluation)
Average time-to-answer	~20 min manual analysis	~2–5 min
Reproducibility of answers	~60%	~95% (test runs)
Responses with full provenance	~40%	~92% (expected)
Analyst effort (manual steps)	High (spreadsheets, queries)	Low (single agent call)
Case study: skill impact analysis	~30 min (manual SLA comparisons)	~6 min (projected)

Table 4 summarizes these preliminary, pilot-level results from CUGA’s evaluation in the Talent Acquisition context. While the figures are based on controlled test environments and limited analyst feedback rather than full production deployment, they highlight the potential of generalist agents to enable substantial efficiency and transparency improvements in enterprise workflows.

Qualitatively, BPO architects noted that CUGA can reduce reliance on ad hoc spreadsheet analysis, provide consistent explanations of sourcing and skill-impact decisions, and support faster onboarding for new team members through step-by-step reasoning grounded in enterprise APIs. Taken together, these preliminary observations indicate that generalist agents such as CUGA hold promise for delivering trustworthy, auditable, and scalable value as they transition from research prototypes toward enterprise-ready systems.

6.3 Qualitative Evidence

To complement quantitative metrics, we highlight two qualitative observations:

Case study. When asked “Which sourcing channel should we prioritize for requisition 05958BR?”, CUGA queried two endpoints (candidate_volume, recommendation_summary), joined on source IDs, and produced a ranked table with SLA metrics. The interface presented not only the recommendation (“LinkedIn”) but also provenance: endpoint names, query parameters, and computation logs. Analysts reported this saved 20–30 minutes of manual dashboard comparisons.

Feedback. BPO architects emphasized reduced “spreadsheet wrangling,” describing CUGA as “freeing time for actual decision-making.” They noted that the ability to decline unsupported requests (e.g., region-level metrics not exposed by APIs) increased their trust, since the agent did not hallucinate unavailable data.

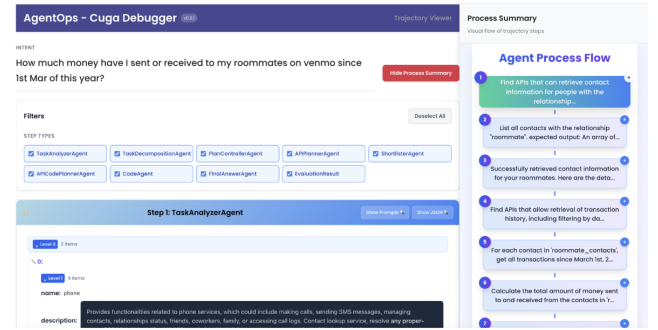


Figure 4: Trajectory Overview

7 Lessons Learned and Insights

The first pilot of CUGA in the BPO Talent Acquisition (TA) context provides early indications that generalist agents can move beyond benchmarks toward enterprise-grade applicability. Phase 1 of the application focused on use cases such as automated candidate scheduling and communication, pipeline visibility, and smart sourcing suggestions. Based on internal projections and controlled simulations, this approach may enable approximately 35% of candidate inquiries to be resolved via self-service and 25% recruiter workflow automation, alongside an estimated 90% reduction in development time and a 50% reduction in development cost compared to task-specific baselines. These preliminary outcomes suggest that generalist agents have the potential to accelerate time-to-value while maintaining the governance and transparency required in enterprise workflows.

From this pilot, we derived a set of lessons and insights—both technical and organizational—that shape the path forward:

Technical insights.

- **Prompt and specification curation.** Minimizing OpenAPI specifications and keeping prompts concise,

schema-grounded, and unambiguous substantially improved reliability and efficiency.

- **Governance alignment.** Restricting to read-only APIs, redacting personally identifiable information, and grounding all answers in canonical definitions were essential to gain organizational trust.
- **Reliability mechanisms.** Interrupt nodes, reflective retries, and a code planner (generating structured pseudo-code) reduced failure rates and improved reproducibility.
- **Monitoring and reproducibility.** Provenance logs and regression testing enabled both audit readiness and systematic debugging of failure modes.
- **Sustainability and extensibility.** The API/Tool Hub streamlined onboarding of new endpoints, allowing rapid iteration as business requirements evolved.
- **Analytical foundation.** A critical advantage was the architecture’s ability to log, monitor, and analyze agent decisions. This enabled systematic investigation of failures and deeper insight into why the agent behaved as it did, laying the groundwork for continuous improvement.

Organizational insights.

- **Human-in-the-loop configuration.** Business users required explicit control over when the agent could act autonomously versus when approval was mandatory, making configurable HITL a central requirement.
- **Benchmarks are not enough.** While AppWorld and WebArena validated general capabilities, enterprise adoption depended on a domain-specific benchmark (BPO-TA) that reflected real recruiter workflows and enabled regression testing.
- **Bridging research and operations.** Deployment success depended as much on organizational alignment—policies, governance, and HITL practices—as on technical breakthroughs. The transition from a promising demo to a trusted production system required deliberate discipline and collaboration across business and research teams.

8 Roadmap

The first phase surfaced clear requirements for the next stage of development. Key technical enhancements now underway include:

- **Configurable HITL.** Fine-grained control over when agents act autonomously or defer to humans.
- **Safety and policy adherence.** As the system expands from read-only analytics to autonomous actions, explicit mechanisms for enforcing enterprise policies and governance are critical.
- **Cost–performance tradeoffs.** Cost and latency should be improved. Static and dynamic short-circuiting is being developed to reduce unnecessary reasoning by adapting the depth of reasoning to task complexity.
- **Trajectory reuse.** Successful execution traces can be generalized into reusable tools, improving both reliability and efficiency when similar tasks recur.

- **Model optimization.** Introducing smaller models for routine subtasks, while reserving large models for complex reasoning, balances accuracy with latency and cost.

A near-term milestone is to extend CUGA’s evaluation to **TauBench2**, which emphasizes policy adherence and human-in-the-loop operation. Success here will further validate CUGA as not only a benchmark-leading agent but also an enterprise-ready system that meets organizational standards of safety and trust.

9 Conclusion

This work provides early evidence that generalist agents can enable measurable business value in enterprise contexts. By combining layered planning, provenance-aware execution, and governance alignment, CUGA can reduce time-to-answer, improve reproducibility, and enable trustworthy automation in talent acquisition. The lessons from Phase 1—both technical and organizational—show that moving from research breakthroughs to enterprise readiness is less about a single algorithmic leap and more about disciplined engineering, governance, and continuous iteration. Architectures like CUGA mark a credible path toward enterprise-ready generalist agents that are safe, efficient, and adaptable.

References

- AI Business. 2022. Alibaba turns to AI to cut customer service costs. Coverage of large-scale customer service automation; Accessed 19-Aug-2025.
- Anthropic. 2024. Introducing computer use, a new Claude 3.5 Sonnet, and more. <https://www.anthropic.com/news/3-5-models-and-computer-use>. Accessed: 2025-08-14.
- Cemri, M.; Pan, M. Z.; Yang, S.; Agrawal, L. A.; Chopra, B.; Tiwari, R.; Keutzer, K.; Parameswaran, A.; Klein, D.; Ramchandran, K.; et al. 2025. Why do multi-agent llm systems fail? *arXiv preprint arXiv:2503.13657*.
- Chen, K.; Cusumano-Towner, M.; Huval, B.; Petrenko, A.; Hamburger, J.; Koltun, V.; and Krähenbühl, P. 2025. Reinforcement learning for long-horizon interactive llm agents. *arXiv preprint arXiv:2502.01600*.
- Chen, Z. 2023. Ethics and discrimination in artificial intelligence-enabled recruitment practices. *Humanities and Social Sciences Communications*, 10(1): 1–13.
- Databricks. 2025. Introducing Genie: AI/BI for the Lakehouse. Accessed 19-Aug-2025.
- de Chezelles, T. L. S.; Gasse, M.; Lacoste, A.; Caccia, M.; Drouin, A.; Boisvert, L.; Thakkar, M.; Marty, T.; Assouel, R.; Shayegan, S. O.; et al. ??? The BrowserGym Ecosystem for Web Agent Research. *Transactions on Machine Learning Research*.
- European Union. 2024. Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). <https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>. OJ L 2024/1689, 12.7.2024.
- Fabris, A.; Baranowska, N.; Dennis, M. J.; Graus, D.; Hacker, P.; Saldivar, J.; Zuiderveen Borgesius, F.; and Biega,

- A. J. 2025. Fairness and Bias in Algorithmic Hiring: A Multidisciplinary Survey. *ACM Transactions on Intelligent Systems and Technology*.
- Fourney, A.; Bansal, G.; Mozannar, H.; Tan, C.; Salinas, E.; Niedtner, F.; Proebsting, G.; Bassman, G.; Gerrits, J.; Alber, J.; et al. 2024. Magentic-one: A generalist multi-agent system for solving complex tasks. *arXiv preprint arXiv:2411.04468*.
- Gupta, S.; Singh, S.; Sabharwal, A.; Khot, T.; and Bogin, B. 2025. Leveraging In-Context Learning for Language Model Agents. *arXiv preprint arXiv:2506.13109*.
- Jace.AI. 2024. jace.ai. <https://jace.ai/blog/awa-1-5-achieves-breakthrough-performance-on-web-arena-benchmark>.
- Jiang, H.; Wu, Q.; Lin, C.-Y.; Yang, Y.; and Qiu, L. ??? LLMingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *The 2023 Conference on Empirical Methods in Natural Language Processing*.
- Kim, G.; Baldi, P.; and McAleer, S. 2023. Language models can solve computer tasks. *Advances in Neural Information Processing Systems*, 36: 39648–39677.
- Kwon, W.; Li, Z.; Zhuang, S.; Sheng, Y.; Zheng, L.; Yu, C. H.; Gonzalez, J.; Zhang, H.; and Stoica, I. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, 611–626.
- LangChain. 2024. LangGraph Documentation: Building Stateful, Multi-Agent Workflows. Accessed 19-Aug-2025.
- Levy, I.; Wiesel, B.; Marreed, S.; Oved, A.; Yaeli, A.; and Shlomov, S. 2024. St-webagentbench: A benchmark for evaluating safety and trustworthiness in web agents. *arXiv preprint arXiv:2410.06703*.
- Marreed, S.; Oved, A.; Yaeli, A.; Shlomov, S.; Levy, I.; Akrabi, O.; Sela, A.; Adi, A.; and Mashkif, N. 2025. Towards enterprise-ready computer using generalist agent. *arXiv preprint arXiv:2503.01861*.
- Microsoft. 2024. Power BI Copilot is now generally available. Accessed 19-Aug-2025.
- Mobile World Live. 2025. Verizon lauds Google Cloud AI customer service move. Accessed 19-Aug-2025.
- Nakano, R.; Hilton, J.; Balaji, S.; Wu, J.; Long, O.; Kim, C.; Hesse, C.; Jain, S.; Kosaraju, V.; Saunders, W.; Jiang, X.; Cobbe, K.; Eloundou, T.; Krueger, G.; Button, K.; Knight, M.; Chess, B.; and Schulman, J. 2021. WebGPT: Browser-assisted question-answering with human feedback. *ArXiv*, abs/2112.09332.
- NYC Department of Consumer and Worker Protection. 2023. NYC Local Law 144 and Final Rules on Automated Employment Decision Tools. <https://www.nyc.gov/assets/dca/downloads/pdf/rules/Rules-Amendment-6RCNY5-300-AEDT.pdf>. Accessed 2025-08-19.
- OpenAI. 2024. OpenAI Swarm: Lightweight Multi-Agent Orchestrator. Accessed 19-Aug-2025.
- OpenAI. 2025. Introducing Operator. <https://openai.com/index/introducing-operator/>. Accessed: 2025-08-14.
- Raghavan, M.; Barocas, S.; Kleinberg, J.; and Levy, K. 2020. Mitigating bias in algorithmic hiring: Evaluating claims and practices. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, 469–481.
- Shen, J.; Jain, A.; Xiao, Z.; Amlekar, I.; Hadji, M.; Podolny, A.; and Talwalkar, A. 2024a. Scribeagent: Towards specialized web agents using production-scale workflow data. *arXiv preprint arXiv:2411.15004*.
- Shen, M.; Li, Y.; Chen, L.; and Yang, Q. 2025. From mind to machine: The rise of manus ai as a fully autonomous digital agent. *arXiv preprint arXiv:2505.02024*.
- Shen, Y.; Song, K.; Tan, X.; Zhang, W.; Ren, K.; Yuan, S.; Lu, W.; Li, D.; and Zhuang, Y. 2024b. Taskbench: Benchmarking large language models for task automation. *Advances in Neural Information Processing Systems*, 37: 4540–4574.
- Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36: 8634–8652.
- Snowflake Engineering. 2024. Introducing Snowflake Cortex Analyst: Natural Language to Accurate SQL. Accessed 19-Aug-2025.
- Tabassi, E. 2023. Artificial Intelligence Risk Management Framework (AI RMF 1.0).
- The Verge. 2025. Verizon adopts Google’s Gemini AI to help customers solve ‘complex’ issues. Accessed 19-Aug-2025.
- Trivedi, H.; Khot, T.; Hartmann, M.; Manku, R.; Dong, V.; Li, E.; Gupta, S.; Sabharwal, A.; and Balasubramanian, N. 2024. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Wu, Q.; Bansal, G.; Zhang, J.; Wu, Y.; Li, B.; Zhu, E.; Jiang, L.; Zhang, X.; Zhang, S.; Liu, J.; Awadallah, A. H.; White, R. W.; Burger, D.; and Wang, C. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*.
- Xie, T.; Zhang, D.; Chen, J.; Li, X.; Zhao, S.; Cao, R.; Hua, T. J.; Cheng, Z.; Shin, D.; Lei, F.; Liu, Y.; Xu, Y.; Zhou, S.; Savarese, S.; Xiong, C.; Zhong, V.; and Yu, T. 2024. OS-World: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- Xu, Q.; Hong, F.; Li, B.; Hu, C.; Chen, Z.; and Zhang, J. 2023. On the Tool Manipulation Capability of Open-source Large Language Models. *arXiv:2305.16504*.
- Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2024. Tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv preprint arXiv:2406.12045*.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*.

Ye, Z.; Lu, S.; Zhang, R.; Fu, J.; et al. 2024. CodeAct: LLMs that Code and Act. *arXiv preprint arXiv:2402.01030*.

Zhang, R.; Qiu, M.; Tan, Z.; Zhang, M.; Lu, V.; Peng, J.; Xu, K.; Agudelo, L. Z.; Qian, P.; and Chen, T. 2025a. Symbiotic cooperation for web agents: Harnessing complementary strengths of large and small llms. *arXiv preprint arXiv:2502.07942*.

Zhang, S.; Yin, M.; Zhang, J.; Liu, J.; Han, Z.; Zhang, J.; Li, B.; Wang, C.; Wang, H.; Chen, Y.; et al. 2025b. Which agent causes task failures and when? on automated failure attribution of llm multi-agent systems. *arXiv preprint arXiv:2505.00212*.

Zhang, W.; Cui, C.; Zhao, Y.; Hu, R.; Liu, Y.; Zhou, Y.; and An, B. 2025c. Agentorchestra: A hierarchical multi-agent framework for general-purpose task solving. *arXiv preprint arXiv:2506.12508*.

Zheng, L.; Yin, L.; Xie, Z.; Sun, C. L.; Huang, J.; Yu, C. H.; Cao, S.; Kozyrakis, C.; Stoica, I.; Gonzalez, J. E.; et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37: 62557–62583.

Zhou, S.; Xu, F. F.; Zhu, H.; Zhou, X.; Lo, R.; Sridhar, A.; Cheng, X.; Ou, T.; Bisk, Y.; Fried, D.; et al. 2023. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *The Twelfth International Conference on Learning Representations*.

A CUGA System Architecture

This section presents the detailed architecture of the CUGA system, including both the Browser and API sub-agents.

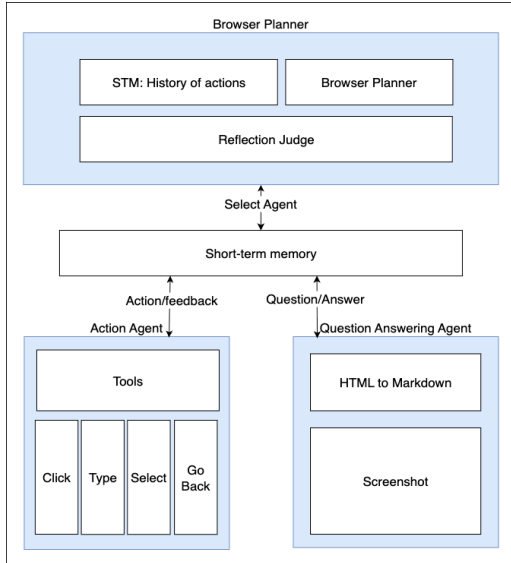


Figure 5: Architecture of the Browser Sub Agent: the Browser Planner combines action history and reflection to select between two execution paths—an Action Agent (click, type, select, navigate) and a Question Answering Agent (DOM-to-Markdown conversion and screenshots).

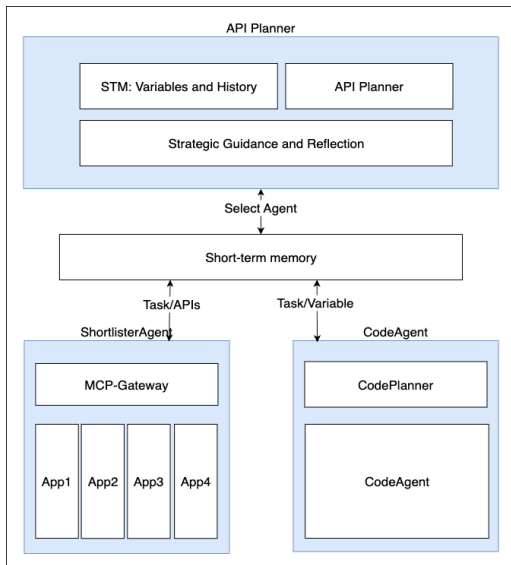


Figure 6: Architecture of the API Sub Agent: the API Planner coordinates short-term memory, reflection, and two executors—the ShortlisterAgent, which routes to APIs through a registry, and the CodeAgent, which generates and executes code via a nested CodePlanner.

B CUGA's Nodes

ChatAgent — Chat

This agent manages follow-up questions and handles conversations that trigger pre-existing, familiar flows.

TaskAnalyzerAgent — Task Analyzer

This agent performs the initial analysis of a user's request to determine its complexity. It decides whether a simple, direct answer is sufficient or if the query requires a more complex, multi-step plan. This agent also identifies related applications based on the user's intent.

TaskDecompositionAgent — Task Decomposition

The agent decomposes complex tasks into smaller, manageable sub-tasks and assigns each to a specialized agent. This approach is essential for effectively solving multi-step problems, as it allows each sub-task to be carefully planned and executed.

PlanControllerAgent — Plan Controller

This agent acts as the central orchestrator of the sub-agents. It reviews the overall plan, tracks the status of sub-tasks created by **TaskDecompositionAgent**, and decides the next steps, ensuring the entire process stays on track from decomposition to the final answer.

Browser Sub-Agent

BrowserPlannerAgent — Browser Planner This agent plans the next steps in natural language, based on its understanding of the DOM page and accompanying images. It then passes tasks to either the **ActionAgent** or **QaAgent**, or decides to conclude the task.

ActionAgent — Action Agent This agent is designed to perform specific subtasks directly on the current web page, such as clicking on an element.

QaAgent — Question Answering Agent This agent is called upon to answer questions related to the current web-page.

Human-in-the-Loop Nodes

SuggestHumanActions — Human Action Suggester

This node is used when human intervention or input would be beneficial. It's necessary for collaborative workflows, allowing the AI to ask for help, clarification, or a decision from the user.

WaitForResponse — Response Waiter

This node pauses the entire workflow until it receives a response, typically from a human user. It works in conjunction with **SuggestHumanActions** to enable true human-in-the-loop processing.

API Sub-Agent

APIPlannerAgent — API Planner

This agent specializes in creating sub-tasks that involve API calls. At each turn, it either calls the **ShortlisterAgent** or **APICodePlannerAgent**, or decides to finish the task and return to the **PlanControllerAgent**. Within this node, there is also a reflection component that runs upon returning from the **CodeAgent**, summarizing the task, checking edge cases, and suggesting strategic recommendations.

APICodePlannerAgent — API Code Planner

This agent, given the shortlisted schema and sub-task decided by the **APIPlannerAgent**, generates a pseudo-natural language plan to guide the coding agent. It can also report missing APIs to the **APIPlannerAgent** as feedback.

CodeAgent — Coding Agent

This agent is responsible for writing and executing code to solve a problem. It is called after the **APICodePlannerAgent**. The generated code is executed in a sandbox to ensure safety, and the result is stored in a variable and fed back to the **APIPlannerAgent** concisely within its context.

ShortlisterAgent — Tool Shortlister

This agent filters and ranks a list of available tools or APIs to find the most relevant ones for a given sub-task generated by the **APIPlannerAgent**.

Other Agents

FinalAnswerAgent — Final Answer

This agent is responsible for gathering the results from all completed tasks and synthesizing them into a final, coherent response for the user. It represents the last step in the cognitive process before presenting the solution.

ReuseAgent — Reuse Agent

This agent is used in "save & reuse" mode, where CUGA suggests that the user save the current autonomous flow into deterministic Python code for safer and more predictable execution. It runs after the **FinalAnswerAgent** in conjunction with human-in-the-loop actions.

C WebArena Benchmark Analysis

The WebArena (Zhou et al. 2023) environment is a realistic and reproducible web environment designed for web agents to interact with. Along with the environment, a benchmark that consists of a set of 812 real-life natural language tasks has been released.

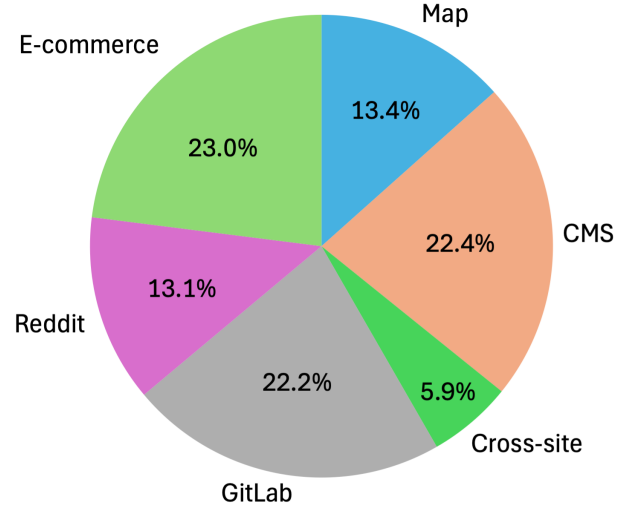


Figure 7: WebArena intent distribution across different websites. Cross-site intents include interaction with multiple websites.

Method	SR (%)
IBM CUGA (ours)	61.7
Operator (OpenAI 2025)	58.1
Autonomous Web Agent (Jace.AI 2024)	57.1
ScribeAgent + GPT-4o (Shen et al. 2024a)	53
AgentSymbiotic (Zhang et al. 2025a)	52.1

Table 5: WebArena Benchmark Leaderboard. SR = Success Rate

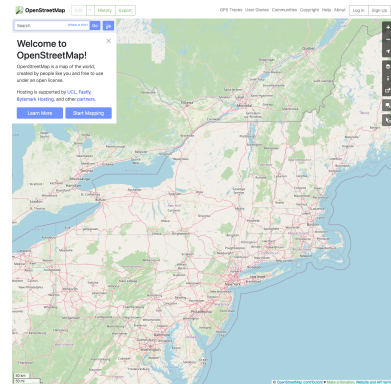


Figure 8: WebArena Map domain task example

D AppWorld Dataset

AppWorld (Trivedi et al. 2024) is a high-quality execution environment designed to test agents with code generation capabilities. The AppWorld benchmark consists of 9 day-to-day application such as Spotify, Amazon Shopping, Gmail, available via 457 APIs and simulating the lives of simulated users. The benchmark is divided into two levels of difficulty - named "Test-Normal" and "Test-Challenge", with the latter being generally more challenging, with each level consisting of several scenario data points, with each scenario consisting of several tasks for an agent to accomplish.

The metrics used to test agents in this benchmark are **Task Goal Completion**, the percentage of tasks for which the agent passed all evaluation tests, and **Scenario Goal Completion**, the percentage of task *scenarios* for which the agent passed all evaluation tests for *all* tasks from that scenario.

	Test-Normal avg (max)	Test-Challenge avg (max)
Num. Apps	1.5 (3)	2.0 (6)
Num. Unique APIs	8.2 (17)	10.5 (26)
Num. API calls	42.5 (244)	46.8 (649)
Num. Sol. Code Lines	41.3 (134)	56.9 (128)
Num. Eval. Tests	5.9 (19)	8.0 (24)
Difficulty Level	1.9 (3)	2.3 (3)

Table 6: AppWorld Benchmark Intent Composition

Method	LLM	Normal TGC (SGC)	Challenge TGC (SGC)
CUGA (ours)	GPT-4.1	73.2 (62.5)	57.6 (48.2)
Chen et al. (2025)	Qwen2.5	72.6 (53.6)	47.2 (28.8)
Gupta et al. (2025)	GPT-4o	68.5 (57.1)	38.9 (23)
ReAct	GPT-4o	48.8 (32.1)	30.2 (13)

Table 7: AppWorld Benchmark Official Leaderboard. TGC and SGC metrics mean task and scenario goal completion respectively.

E BPO-TA Benchmark Details

This section provides comprehensive details about the BPO-TA (Business Process Operations - Talent Acquisition) benchmark, including task taxonomy, catalog, and example queries.

E.1 Task Categories and Examples

E.2 Dataset Taxonomy

Figure 9 presents a taxonomy of the dataset catalog, providing a structured overview of the categories represented in our benchmark. The root node ("Dataset Taxonomy") splits into five major categories:

Source Analytics: Queries that focus on requisition sources, including SLA-based ranking, total hires by source, candidate volumes, funnel conversion performance, and weighted prioritization (IDs 2–5, 8).

Skill Analytics: Queries that test reasoning about the impact of skills on SLA performance, fill rates, and relevance judgments, as well as provenance through models and data sources used in calculations (IDs 7, 10, 17, 18, 20, 21, 22).

Methodology & Metadata: Queries targeting definitional and contextual knowledge, such as SLA definitions, sample sizes, evaluation metrics, timeframe coverage, and averages over requisitions (IDs 14–16, 27, 33).

Error Handling & Clarification: Queries that probe conversational robustness, such as missing required parameters and invalid job IDs (IDs 24, 31).

Unsupported / Future Capabilities: Queries intentionally designed to highlight system limitations and possible extensions, including job description optimization, hiring manager analytics, full funnel counts, per-source time-to-fill, geographic filters, SLA countdowns, and full job-card details (IDs 23, 25–26, 28–30, 32).

The visualization complements the full catalog table by showing at a glance how queries are distributed across categories. It highlights the dataset’s design principle: to balance supported tasks (sources, skills, metadata) with robustness testing (error handling) and aspirational use cases (unsupported/future capabilities).

E.3 Complete Task Catalog

Table 9 lists the full set of 26 BPO-TA tasks, including end-point usage, glue code requirements, and number of calls. This catalog is intended to ensure reproducibility and provide a reference for ablations.

F Experimental Results and Trajectories

This section presents detailed experimental results and example system trajectories demonstrating CUGA’s performance across different benchmarks.

Category	Example Query
Lookup	“How is the SLA metric defined for 05958BR?”
Join	“For requisitions like 05958BR, which sources provided the most candidates, and how effective were they at converting to hires?”
Looped comparison	“Out of Python, Quantum Physics, Cyber Engineering, Risk Analysis, Wireshark – which skills negatively affect SLA?”
Provenance explanation	“What models and datasets were used to compute the SLA impact of Python?”
Graceful failure	“Who’s the hiring manager for 05959BR and how responsive is she?”

Table 8: BPO-TA benchmark categories with representative queries.

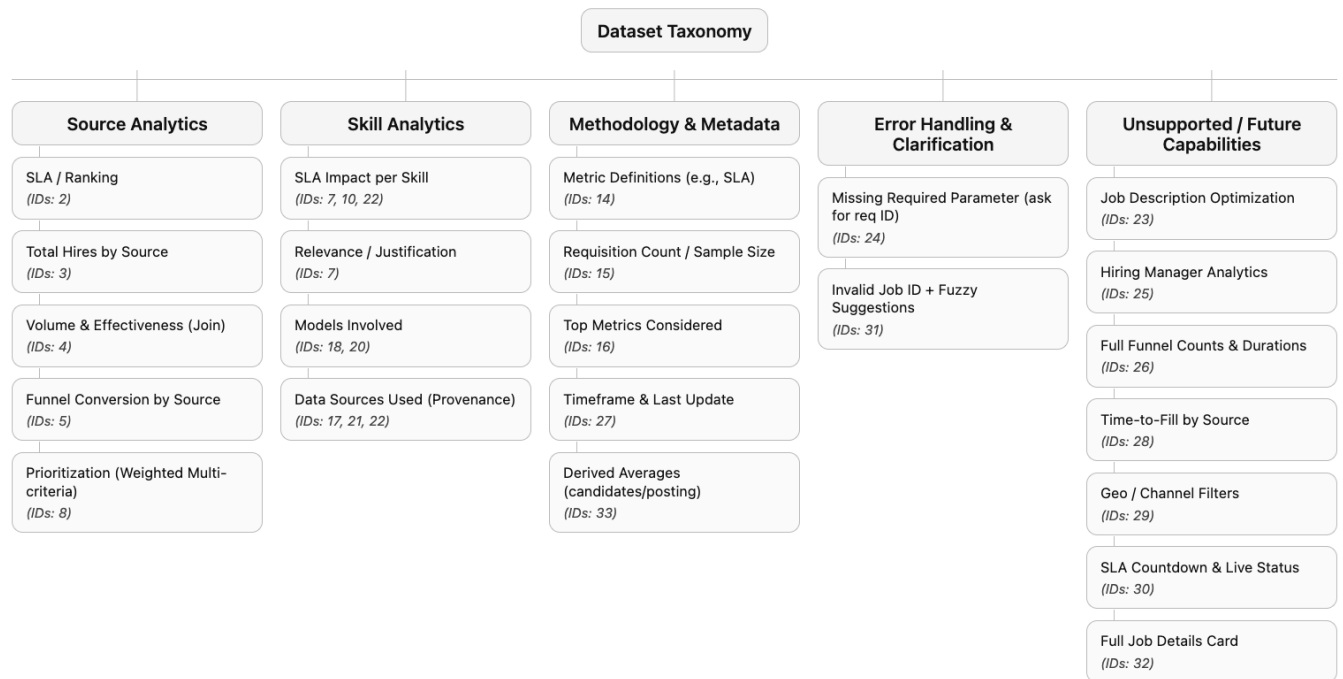


Figure 9: Dataset taxonomy visualization. Each branch corresponds to a query category (e.g., Source Analytics, Skill Analytics), and leaf nodes list the dataset entry IDs that belong to that category. The visualization highlights the breadth of query types covered by the benchmark, ranging from source- and skill-focused analytics to methodology and error-handling cases, as well as unsupported or future capabilities.

ID	Task (Simplified Query)	Endpoints Used	Glue Code / Reasoning	Calls
1	Define SLA metric for requisition 05958BR	get_definitions	None	1
2	Lowest-performing source by SLA for 05958BR	recommendation.summary	Rank SLA values	1
3	Which source had most candidates and how effective?	candidate.volume, recommendation.summary	Join on source name	2
4	Does Python impact SLA?	skill.impact, recommendation.summary	Aggregate by skill	2
5	Compare SLA impact of multiple skills (looped)	skill.impact	Iterative calls per skill, filter negative values	5+
6	Which skill most improves SLA?	skill.impact	Max delta computation	3+
7	Funnel conversion for 05958BR	funnel.conversion	Extract ratios	1
8	Candidate drop-off by stage for 05958BR	funnel.conversion	Compute differences per stage	1
9	Top source by time-to-fill (05958BR)	recommendation.summary	Sort by TTF metric	1
10	Cross-source SLA comparison (3 sources)	recommendation.summary	Filter + compare SLA side-by-side	1
11	Explain provenance of Python SLA impact	skill.impact, get_definitions	Surface dataset + model metadata	2
12	Show data sources used for candidate counts	candidate.volume, get_definitions	Join with provenance metadata	2
13	Fill rate for source "LinkedIn" on 05958BR	recommendation.summary	Lookup specific source	1
14	Compare conversion rate across 3 requisitions	funnel.conversion	Aggregate across requisitions	3
15	Most stable source across requisitions	recommendation.summary	Variance analysis across SLA values	5+
16	Skill that worsens SLA most	skill.impact	Min delta computation	3+
17	List all requisitions with SLA $\geq$ 70%	recommendation.summary	Filter on threshold	1
18	Average SLA by region (unsupported)	None	Must decline gracefully	0
19	Hiring manager responsiveness for 05959BR (unsupported)	None	Must decline gracefully	0
20	Impact of Wireshark on SLA	skill.impact	Single lookup	1
21	Rank skills by SLA impact for 05958BR	skill.impact	Sort descending	5+
22	Explain SLA computation formula	get_definitions	Return narrative explanation	1
23	Detect missing data for requisition 05959BR	recommendation.summary	Error handling / graceful failover	1
24	Compare top 5 sources for SLA	recommendation.summary	Sort + slice top-N	1
25	Which skill reduces time-to-fill?	skill.impact	Compare deltas	3
26	Unsupported cross-app question (mixing HR + sourcing)	None	Must decline gracefully	0

Table 9: BPO-TA catalog: 26 tasks with associated endpoints, glue code, and number of calls. IDs marked "unsupported" test graceful degradation.

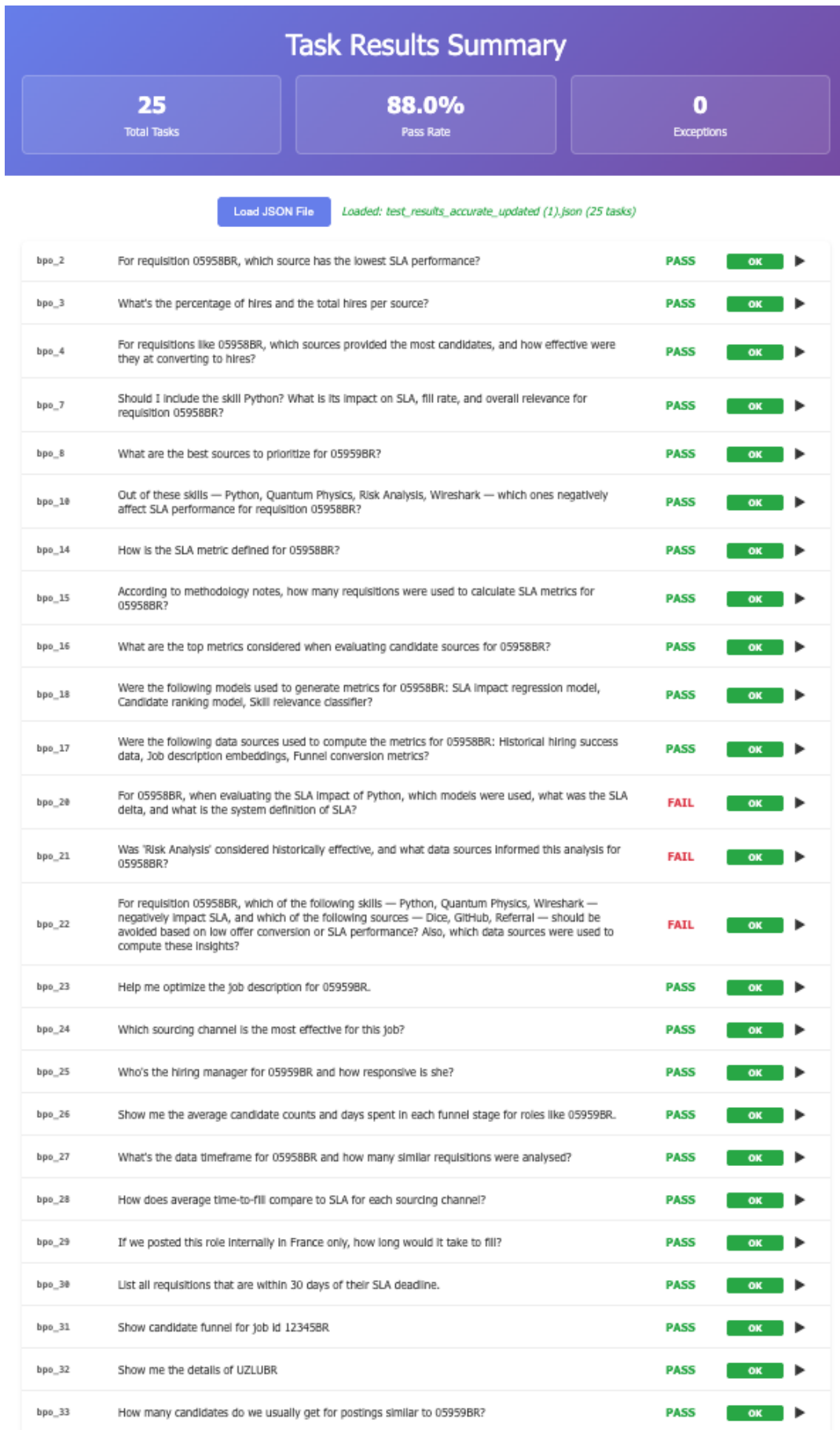


Figure 10: Detailed performance results on the BPO-TA benchmark showing CUGA's effectiveness across different task categories.

Figure 11: Detailed results on BPO benchmark - Part 1

```

1 ;[
2 {
3   ID: 2,
4   Utterance:
5     'For requisition 05958BR, which source has the lowest SLA performance?',
6   Output: 'CyberSec Jobs with 67%',
7   Explanation:
8     'CyberSec Jobs was identified as the lowest-performing source because its SLA success rate is 67 %, well
9     below Dice (80 %), LinkedIn (79 %), GitHub (78 %), and the other sources returned by the API.',
10  'Expected Response Summary': 'Lists sources ranked by SLA success rate.',
11  'APIs Used': '/tools/candidate-source/sla-per-source/',
12  'Glue Code': 'None',
13  'Number of API Calls': '1',
14  'Special Note': '',
15 },
16 {
17   ID: 3,
18   Utterance:
19     "What's the percentage of hires and the total hires per source for requisition 05958BR?",
20   Output:
21     'Dice: 11 hires (27.5%), GitHub: 10 hires (25%), LinkedIn: 7 hires (17.5%), Internal: 5 hires (12.5%),
22     Referral: 4 hires (10%), CyberSec Jobs: 3 hires (7.5%)',
23   Explanation:
24     "Total hires across all sources sum to 40; each source's percentage was calculated as (hires ÷ 40) × 100
25     and the list was ordered from highest to lowest share.",
26   'Expected Response Summary':
27     'Provides both total hire count and percentage per candidate source.',
28   'APIs Used': '/tools/candidate-source/total-hires-by-source/',
29   'Glue Code':
30     'For each source: percent = total_hires / sum(total_hires) × 100. Return sorted list with both values.',
31   'Number of API Calls': '1',
32   'Special Note': '',
33 },
34 {
35   ID: 4,
36   Utterance:
37     'For requisitions like 05958BR, which sources provided the most candidates, and how effective were they
38     at converting to hires?',
39   Output:
40     'LinkedIn: 505 candidates (18%), 7 hires. Offer acceptance rate: 70%. Dice: 501 candidates (18%), 11
41     hires. Offer acceptance rate: 79%. GitHub: 453 candidates (16%), 10 hires. Offer acceptance rate: 77%.',
42   Explanation:
43     'Candidate counts and percentages were taken from the candidate-volume API; hire counts and offer-
44     acceptance rates were taken from the recommendation-summary API. The two tables were joined on "source_name",
45     producing a combined view of volume and effectiveness for the three leading sources.',
46   'Expected Response Summary':
47     "Shows each source's candidate volume and offer/hire success metrics for jobs similar to 05958BR.",
48   'APIs Used':
49     '/tools/candidate-source/candidate-volume-by-source/, /tools/candidate-source/source-recommendation-
50     summary/',
51   'Glue Code':
52     'Match source names across APIs. Return volume (count + percent) and offer acceptance rate / hire
53     count.',
54   'Number of API Calls': '2',
55   'Special Note':
56     "Cross-references performance and volume per source. Requires joining APIs on 'source_name'.",
57 },
58 ]

```



```

49 {
50     ID: 5,
51     Utterance:
52         'Did Dice provide a good funnel conversion rate for requisition 05958BR?',
53     Output:
54         'Dice had a strong funnel: 11% of candidates reached first-round review, 6.8% reached interviews, and
55         2.7% received offers. It also resulted in 11 hires – 27.5% of all hires, making it one of the top-performing
56         sources.',
57     Explanation:
58         "Stage-wise funnel percentages for Dice were fetched from the funnel-conversion API; hire count came
59         from total-hires API. Comparing 11 hires to the overall 40-hire total (27.5 %) confirmed Dice's strong end-to-
60         end performance.",
61     'Expected Response Summary':
62         "Shows Dice's candidate review, interview, and offer percentages along with total hire count.",
63     'APIs Used':
64         '/tools/candidate-source/funnel-conversion-by-source/, /tools/candidate-source/total-hires-by-source/',
65     'Glue Code':
66         'Join funnel metrics and hire count for Dice. Calculate hire ratio from total and include contextual
67         summary of performance.',
68     'Number of API Calls': '2',
69     'Special Note':
70         "Highlights both stage-wise funnel and end result to justify 'good' performance.",
71 },
72 {
73     ID: 7,
74     Utterance:
75         'Should I include the skill Python? What is its impact on SLA, fill rate, and overall relevance for
76         requisition 05958BR?',
77     Output:
78         'Python has no impact on SLA (90% with and without). Fill rate is slightly improved (1.4% vs 1.3%) and
79         time to fill is shorter (70 vs 86 days). However, it is not flagged as a relevant skill in historical data,
80         and appears in both successful and unsuccessful postings.',
81     Explanation:
82         'The SLA-impact API showed identical 90 % SLA with and without Python. Fill-rate API indicated a
83         marginal improvement and shorter time-to-fill. The relevance-justification API marked Python as neutral.
84         Aggregating these results demonstrates Python is harmless but not a differentiator.',
85     'Expected Response Summary':
86         'Assesses whether Python should be included by evaluating SLA impact, fill rate trends, and relevance
87         label.',
88     'APIs Used':
89         '/tools/skills/skill-impact-sla/, /tools/skills/skill-impact-fill-rate/, /tools/skills/skill-relevance-
90         justification/',
91     'Glue Code':
92         "Aggregate SLA and fill metrics from 'sla' and 'fill-rate' APIs. Use relevance flag and explanation from
93         'skill-relevance-justification'.",
94     'Number of API Calls': '3',
95     'Special Note':
96         'Intentionally spans multiple APIs to validate both statistical and semantic relevance of a skill.',
97 },

```

```

85  {
86      ID: 8,
87      Utterance: 'What are the best sources to prioritize for 05959BR?',
88      Output:
89          'You should prioritize Dice, GitHub, and LinkedIn. Dice and LinkedIn both met SLA 100% of the time and
brought in 18% of all candidates. Dice had a strong offer conversion rate (2.7%), and GitHub had the highest
conversion (2.8%) despite slightly lower SLA. Indeed should be avoided due to 0% SLA and 0% offer
conversion.',
90      Explanation:
91          'Each source received a weighted score (50 % SLA success, 30 % candidate volume share, 20 % offer-
conversion rate). Dice and LinkedIn tied for top SLA (100 %) and high volume; GitHub's best-in-class
conversion (2.8 %) offset its 80 % SLA. Indeed scored 0 on SLA and offers, so it was excluded.',
92      'Expected Response Summary':
93          'Recommends top-performing sources by combining SLA success, candidate volume, and funnel
effectiveness.',
94      'APIs Used':
95          '/tools/candidate-source/sla-per-source/, /tools/candidate-source/candidate-volume-by-source/, /tools/
candidate-source/funnel-conversion-by-source/',
96      'Glue Code':
97          'Aggregate data by source across APIs. Compute a weighted ranking based on SLA %, candidate volume, and
offer conversion rate. Return top 3 and justify exclusions.',
98      'Number of API Calls': '3',
99      'Special Note':
100         'This benchmark tests multi-criteria decision-making and cross-API synthesis.',
101  },
102  {
103      ID: 10,
104      Utterance:
105          'Out of these skills – Python, Quantum Physics, Risk Analysis, Wireshark – which ones negatively affect
SLA performance for requisition 05958BR?',
106      Output:
107          'Wireshark negatively affected SLA performance and it reduced SLA from 85% to 70%. The rest of the
skills had neutral or no measurable impact.',
108      Explanation:
109          'The SLA-impact API was queried for each skill. Skills whose SLA with the skill was lower than without
it were flagged as negative: Cyber Engineering (-15 pp) and Wireshark (-15 pp). Other skills showed 0 pp
change.',
110      'Expected Response Summary':
111          'Loops through the provided list of skills, calling SLA impact API for each, and returns only those that
reduce SLA.',
112      'APIs Used': '/tools/skills/skill-impact-sla/',
113      'Glue Code':
114          'For each skill in the list, call the API and compare 'sla_achievement_with_skill' vs.
'sla_achievement_without_skill'. Collect and return skills where the value decreases.',
115      'Number of API Calls': 'N (loop)',
116      'Special Note':
117          'Tests the agent's ability to perform iterative API calls and reason over numeric comparisons across
multiple results.',
118  },
119  {
120      ID: 14,
121      Utterance: 'How is the SLA metric defined for 05958BR?',
122      Output:
123          'SLA is defined as 'Percentage of candidates reviewed within the defined SLA window (e.g., 48 hours)'.',
124      Explanation:
125          'The definitions-and-methodology endpoint contains a JSON field "sla" holding the textual definition;
the agent extracted that string verbatim.',
126      'Expected Response Summary':
127          'Returns the definition of the SLA metric for the given requisition.',
128      'APIs Used': '/tools/candidate-source/definitions-and-methodology/',
129      'Glue Code': 'Extract the 'sla' field from the 'definitions' section.',
130      'Number of API Calls': '1',
131      'Special Note':
132          'Tests the agent's ability to locate and return a specific definition.',
133  },

```

```

134 {
135     ID: 15,
136     Utterance:
137         'According to methodology notes, how many requisitions were used to calculate SLA metrics for 05958BR?',
138     Output: 'Metrics are calculated from 1047 requisitions.',
139     Explanation:
140         'The methodology response includes a note like "Metrics calculated over N = 1047 requisitions"; the
agent parsed the integer 1047 and returned it.',
141     'Expected Response Summary':
142         'Returns the number of requisitions used to compute the reported metrics.',
143     'APIs Used': '/tools/candidate-source/definitions-and-methodology/',
144     'Glue Code': "Extract requisition count from 'calculation_notes' string.",
145     'Number of API Calls': '1',
146     'Special Note':
147         'Tests string parsing / information extraction from notes field.',
148 },
149 {
150     ID: 16,
151     Utterance:
152         'What are the top metrics considered when evaluating candidate sources for 05958BR?',
153     Output:
154         'The top metrics considered are: SLA %, First round review %, Offer acceptance rate, Candidate volume,
Total hires.',
155     Explanation:
156         'The agent read the "top_metrics_considered" array from the methodology API response and returned the
metrics in the same order.',
157     'Expected Response Summary':
158         'Returns the list of top metrics considered for source evaluation.',
159     'APIs Used': '/tools/candidate-source/definitions-and-methodology/',
160     'Glue Code':
161         "Extract and return 'top_metrics_considered' array as a formatted list.",
162     'Number of API Calls': '1',
163     'Special Note': 'Tests structured list extraction and formatting.',
164 },
165 {
166     ID: 18,
167     Utterance:
168         'Were the following models used to generate metrics for 05958BR: SLA impact regression model, Candidate
ranking model, Skill relevance classifier?',
169     Output:
170         "Yes, 'SLA impact regression model' and 'Skill relevance classifier' were used. 'Candidate ranking
model' was not listed among the models involved.",
171     Explanation:
172         'The agent compared each provided model name against the "models_involved" array returned by data-
sources-used API and reported matches (used) or non-matches (not used).',
173     'Expected Response Summary':
174         'Loops through the provided list of models and reports which ones were used.',
175     'APIs Used': '/tools/skills/data-sources-used/',
176     'Glue Code':
177         "For each provided model name, check if it appears in 'models_involved' array. Return results per
model.",
178     'Number of API Calls': '1',
179     'Special Note':
180         'Tests loop-based reasoning and partial matching for list membership.',
181 },

```

```

182 {
183     ID: 17,
184     Utterance:
185         'Were the following data sources used to compute the metrics for 05958BR: Historical hiring success
data, Job description embeddings, Funnel conversion metrics?',
186     Output:
187         "Yes, 'Historical hiring success data' and 'Funnel conversion metrics' were used. 'Job description
embeddings' was not listed among the data sources.",
188     Explanation:
189         'Each candidate data source was checked against the "datasets_used" array from data-sources-used API;
two matched and one did not, which the agent reported accordingly.',
190     'Expected Response Summary':
191         'Loops through the provided list of data sources and reports which ones were used.',
192     'APIs Used': '/tools/skills/data-sources-used/',
193     'Glue Code':
194         "For each provided data source name, check if it appears in 'datasets_used' array. Return results per
source.",
195     'Number of API Calls': '1',
196     'Special Note':
197         'Tests loop-based reasoning and partial matching for list membership.',
198 },
199 {
200     ID: 20,
201     Utterance:
202         'For 05958BR, when evaluating the SLA impact of Python, which models were used, what was the SLA delta,
and what is the system definition of SLA?',
203     Output:
204         "'SLA impact regression model' was used. The SLA delta for Python was 0.0%. SLA is defined as
'Percentage of candidates reviewed within the defined SLA window (e.g., 48 hours)'." ,
205     Explanation:
206         "The SLA-impact API showed a 0 % delta for Python; data-sources-used API confirmed the 'SLA impact
regression model' was involved; the methodology API supplied the formal SLA definition. These three pieces
were combined into one coherent answer.",
207     'Expected Response Summary':
208         'Combines model lookup, retrieves actual SLA delta, and returns SLA definition.',
209     'APIs Used':
210         '/tools/skills/skill-impact-sla/, /tools/skills/data-sources-used/, /tools/candidate-source/definitions-
and-methodology/',
211     'Glue Code':
212         'Retrieve SLA impact for Python → retrieve models involved → retrieve SLA definition → synthesize into
single response.',
213     'Number of API Calls': '3',
214     'Special Note':
215         'Agent must combine numerical result (delta), model lookup, and formal definition into unified answer.',
216 },

```

```

217 {
218     ID: 21,
219     Utterance:
220         "Was 'Risk Analysis' considered historically effective, and what data sources informed this analysis for
05958BR?",
221     Output:
222         "'Risk Analysis' is **not** considered effective: historical analysis shows it is correlated with a
**highly negative impact on SLA**. The evaluation used these data sources: Historical hiring success data,
Requisition skill tagging, Funnel conversion metrics, and Candidate quality feedback.",
223     Explanation:
224         'The skill-analysis API flagged Risk Analysis as negatively correlated with SLA. The data-sources-used
API listed the four datasets underpinning the evaluation, and both results were consolidated in the
response.',
225     'Expected Response Summary':
226         'States that Risk Analysis negatively affects SLA and lists the datasets that informed the analysis.',
227     'APIs Used':
228         '/tools/skills/skill-analysis/, /tools/skills/data-sources-used/',
229     'Glue Code':
230         "Look for 'Risk Analysis' in skill-analysis → read correlation; call data-sources-used to list
datasets.",
231     'Number of API Calls': '2',
232     'Special Note':
233         "Correlation wording corrected to match API ('highly negative impact on SLA').",
234 },
235 {
236     ID: 22,
237     Utterance:
238         'For requisition 05958BR, which of the following skills – Python, Quantum Physics, Wireshark –
negatively impact SLA, and which of the following sources – Dice, GitHub, Referral – should be avoided based
on low offer conversion or SLA performance? Also, which data sources were used to compute these insights?',
239     Output:
240         "None of the listed skills show a negative SLA impact in the available data: Python's SLA delta is 0 %,
and Quantum Physics and Wireshark are not present in the analysis. Likewise, Dice (SLA 100 %, offer-acceptance
79 %), GitHub (SLA 80 %, offer-acceptance 77 %), and Referral (SLA 100 %, offer-acceptance 67 %) all perform
satisfactorily, so none of them need to be avoided. These insights were produced using the following datasets:
Historical hiring success data, Requisition skill tagging, Funnel conversion metrics, and Candidate quality
feedback.",
241     Explanation:
242         'The agent looped over each skill, querying SLA-impact and skill-analysis APIs; none yielded a negative
delta. It then looped over each source, combining SLA-per-source and funnel-conversion data; all three met
both SLA and offer benchmarks, so none were flagged. Finally, the datasets_used array was fetched once and
reported.',
243     'Expected Response Summary':
244         'Explains that no provided skills hurt SLA, no provided sources require avoidance, and lists datasets
used.',
245     'APIs Used':
246         '/tools/skills/skill-impact-sla/, /tools/skills/skill-analysis/, /tools/skills/data-sources-used/, /
tools/candidate-source/funnel-conversion-by-source/, /tools/candidate-source/sla-per-source/',
247     'Glue Code':
248         'Loop over skills → check SLA delta / analysis; loop over sources → read SLA % & offer-acceptance; fetch
datasets_used once.',
249     'Number of API Calls': '5 APIs (multiple calls due to loops)',
250     'Special Note':
251         'Fully aligned with current API payloads–no unsupported claims.',
252 },

```

```

253 {
254     ID: 23,
255     Utterance: 'Help me optimize the job description for 05959BR.',
256     Output:
257         'Sorry – the available APIs do not expose the current job-description text, so I cannot directly
optimise it.',
258     HypotheticalOutput:
259         '* Remove “Quantum Physics” (0 occurrences in 40 historical successful jobs).\n• Replace “Cyber
Engineering” with the broader “Security Engineering” to widen the talent pool.\n• Move the soft-skills section
higher in the ad to improve engagement.\nEstimated reduction in time-to-fill: ≈ 6 days.',
260     Explanation:
261         'No endpoint returns or accepts free-text job descriptions, so optimisation is unsupported. The
hypothetical section shows what the answer could look like if such an endpoint were added.',
262     'Expected Response Summary':
263         'Apologises for missing capability; optionally provides an illustrative optimisation if the feature
existed.',
264     'APIs Used': 'None',
265     'Glue Code': 'N/A',
266     'Number of API Calls': '0',
267     'Special Note':
268         'Demonstrates graceful degradation plus a forward-looking example.',
269 },
270 {
271     ID: 24,
272     Utterance: 'Which sourcing channel is the most effective for this job?',
273     Output:
274         "Sure, I can help with that. Could you please provide the requisition ID you're interested in?",
275     HypotheticalOutput: '',
276     Explanation:
277         'Illustrates a clarification turn when a required parameter (requisition_id) is missing.',
278     'Expected Response Summary':
279         'Prompts the user for the missing job ID instead of guessing.',
280     'APIs Used': 'None',
281     'Glue Code': 'N/A',
282     'Number of API Calls': '0',
283     'Special Note': 'Tests conversational error-handling with zero API usage.',
284 },
285 {
286     ID: 25,
287     Utterance:
288         "Who's the hiring manager for 05959BR and how responsive is she?",
289     Output:
290         "Sorry – current APIs don't return hiring-manager names or responsiveness metrics, so I can't answer
that.",
291     HypotheticalOutput:
292         'Emma Johnson is the hiring manager. She reviews candidates within 2 days on average (90th-percentile
responsiveness).',
293     Explanation:
294         'No endpoints expose hiring-manager data; a possible future answer is sketched for context.',
295     'Expected Response Summary':
296         'Explains unsupported request; shows what a successful answer might include.',
297     'APIs Used': 'None',
298     'Glue Code': 'N/A',
299     'Number of API Calls': '0',
300     'Special Note': 'Covers gap #3: hiring-manager analytics.',
301 },

```

```

302 {
303     ID: 26,
304     Utterance:
305         'Show me the average candidate counts and days spent in each funnel stage for roles like 05959BR.',
306     Output:
307         "I'm sorry, but none of the available APIs provide stage-by-stage candidate counts or time-in-status
metrics, so I can't generate a funnel table.",
308     HypotheticalOutput:
309         'In Progress: 73 candidates, 1 day • Under Consideration: 73, 1 day • Recruiter Screen: 43, 1 day • HM
Submit: 8, 1 day • HM Review: 8, 2 days • Interviewing: 5, 2 days • Interviewed: 5, 2 days • Offer: 1, 2 days
• Offer Accept: 1, 3 days • Hired: 1, 2 days • Pre-Boarding: 1, 3 days • Pre-Boarding Complete: 1, 2 days.',
310     Explanation:
311         'The existing funnel-conversion API only returns percentages by source, not absolute counts or durations
per stage.',
312     'Expected Response Summary':
313         'States limitation; optional mock table shows desired granularity.',
314     'APIs Used': 'None',
315     'Glue Code': 'N/A',
316     'Number of API Calls': '0',
317     'Special Note': 'Covers gap #4: full funnel metrics.',
318 },
319 {
320     ID: 27,
321     Utterance:
322         "What's the data timeframe for 05958BR and how many similar requisitions were analysed?",
323     Output:
324         'The metrics cover 23 Oct 2023 – 15 Mar 2025 (= 18 months) and were last updated on 29 Apr 2025. A total
of 39 similar requisitions were analysed.',
325     HypotheticalOutput: '',
326     Explanation:
327         'Pulled time_frame_start, time_frame_end, data_last_updated and total_requisitions_analysed from the
metadata-and-timeframe endpoint.',
328     'Expected Response Summary':
329         'Returns start/end dates, last update date, and requisition count.',
330     'APIs Used': '/tools/candidate-source/metadata-and-timeframe/',
331     'Glue Code': 'Read JSON fields and format into a concise sentence.',
332     'Number of API Calls': '1',
333     'Special Note':
334         'Demonstrates a fully supported recency / frequency query (gap #5).',
335 },
336 {
337     ID: 28,
338     Utterance:
339         'How does average time-to-fill compare to SLA for each sourcing channel?',
340     Output:
341         "I'm afraid I can't provide that comparison because the current APIs don't expose time-to-fill broken
down by source.",
342     HypotheticalOutput:
343         'Dice: 74 days vs SLA 74 → on target • GitHub: 76 vs 74 → +2 days • CyberSec Jobs: 89 vs 86 → +3 days •
Internal: 71 vs 69 → +2 days • LinkedIn: 80 vs 80 → on target • Referral: 69 vs 69 → on target.',
344     Explanation:
345         'Only the definition endpoint describes *what* time-to-fill means; no endpoint provides per-source
values to compute deltas.',
346     'Expected Response Summary':
347         'Gracefully declines unsupported analysis; shows sample comparative table.',
348     'APIs Used': 'None',
349     'Glue Code': 'N/A',
350     'Number of API Calls': '0',
351     'Special Note': 'Covers gap #6.',
352 },

```



```

353 {
354   ID: 29,
355   Utterance:
356     'If we posted this role internally in France only, how long would it take to fill?',
357   Output:
358     "Current APIs don't allow filtering by geography or posting channel, so I'm unable to estimate a France-
only internal posting timeline.",
359   HypotheticalOutput:
360     'Average time-to-fill for internal-only postings in France: 65 days (-6 days vs global average).',
361   Explanation:
362     'No endpoints accept country/channel parameters; the example illustrates desired behaviour.',
363   'Expected Response Summary':
364     'Explains unsupported geographic/channel filter and gives mock estimation.',
365   'APIs Used': 'None',
366   'Glue Code': 'N/A',
367   'Number of API Calls': '0',
368   'Special Note': 'Covers gap #7: region & channel-specific analytics.',
369 },
370 {
371   ID: 30,
372   Utterance:
373     'List all requisitions that are within 30 days of their SLA deadline.',
374   Output:
375     "Sorry – the API suite doesn't provide live requisition status or SLA countdowns, so I can't generate
that list.",
376   HypotheticalOutput:
377     'UZLUBR (Sourcing Manager) – 27 days remaining • F50HBR (Offering Manager) – 18 days • MJZ1BR (Offering
Manager) – 12 days.',
378   Explanation:
379     'No endpoint surfaces open requisitions with SLA dates. The hypothetical section demonstrates the
desired list format.',
380   'Expected Response Summary':
381     'States limitation and shows example list of near-deadline requisitions.',
382   'APIs Used': 'None',
383   'Glue Code': 'N/A',
384   'Number of API Calls': '0',
385   'Special Note': 'Covers gap #8 and introduces a potential future endpoint.',
386 },
387 {
388   ID: 31,
389   Utterance: 'Show candidate funnel for job id 12345BR',
390   Output:
391     'No job can be found with the ID 12345BR.\nDid you want to use one of the following job IDs instead?\n•
UZLUBR – Sourcing Manager\n• F50HBR – Offering Manager\n• MJZ1BR – Offering Manager\n• 5TTKBR – Delivery
Analyst',
392   Explanation:
393     'Because 12345BR does not exist, the assistant returns a polite error plus four close-match IDs
(simulating fuzzy search in the ATS). No API call is made for a bad ID.',
394   'Expected Response Summary':
395     'Graceful 'ID not found' message with suggested alternatives.',
396   'APIs Used': 'None',
397   'Glue Code': 'N/A',
398   'Number of API Calls': '0',
399   'Special Note': 'Error-handling scenario for invalid requisition IDs.',
400 },

```

```

401 {
402     ID: 32,
403     Utterance: 'Show me the details of UZLUBR',
404     Output:
405         "Sorry – none of the current APIs provide full job-card details (title, location, hiring-manager email,
406         etc.), so I can't display that information.",
407     HypotheticalOutput:
408         'Title: Sourcing Manager • Status: Open • City: New York • Posted: 21 Nov 2024 • SLA Deadline: 20 Jan
409         2025 • Hiring Manager: Sophia Jones (sophiajones@somecompany.com)',
410     Explanation:
411         'There is no /job-details/ endpoint. The hypothetical section illustrates what the response would look
412         like if such an endpoint became available.',
413     'Expected Response Summary':
414         'Explains unsupported request and sketches desired output.',
415     'APIs Used': 'None',
416     'Glue Code': 'N/A',
417     'Number of API Calls': '0',
418     'Special Note': "Completes coverage for full requisition card' requests.",
419 },
420 {
421     ID: 33,
422     Utterance:
423         'How many candidates do we usually get for postings similar to 05959BR?',
424     Output: 'On average, similar postings attract **73 candidates**.',
425     Explanation:
426         'candidate-volume-by-source returns `total_candidate_volume = 2828`; metadata-and-timeframe shows
427         `total_requisitions_analysed = 39`. Dividing 2828 ÷ 39 ≈ 73 yields the average.',
428     'Expected Response Summary':
429         'Returns average candidate count for comparable requisitions.',
430     'APIs Used':
431         '/tools/candidate-source/candidate-volume-by-source/, /tools/candidate-source/metadata-and-timeframe/',
432     'Glue Code':
433         'avg_candidates = total_candidate_volume / total_requisitions_analysed',
434     'Number of API Calls': '2',
435     'Special Note': 'Covers the repeated average candidate volume questions.',
436 },
437 ]
438

```

Figure 12: CUGA performing the task: "For each product type in my Amazon cart, buy the cheapest product and move the rest to the wish list." The reflection API agent then summarized the progress so far and recommended moving non-cheapest products from the wish list as well, to avoid accidentally purchasing them later. This way, the agent doesn't mistakenly buy products that are not the cheapest option.

Step 16: APIPlannerAgent

Show Prompts

Show JSON

Overall Status/Analysis: The CoderAgent has successfully completed all preparatory data processing steps for your task. It retrieved your Amazon cart contents, enriched each item with its product type and price, and correctly identified the cheapest product per type. The resulting variable, `cart_cheapest_selection`, contains two arrays: one with the `product_ids` to keep (the cheapest per product type) and one with the `product_ids` to move to your wish list (all others). The logic and output are consistent with your requirements.

Summary of Progress:

- All cart items were retrieved and stored in `cart_contents`.
- Each item's `product_type` and price were obtained and stored in `cart_product_details`.
- The cheapest product per `product_type` was identified, and arrays for products to keep and to move were created in `cart_cheapest_selection`.

Strategic Recommendation: The next step is to take action based on this selection:

- Move all `product_ids` in `product_ids_to_move` from your cart to your wish list (handling each move individually if required by the API).
- After moving, verify that only the intended items remain in your cart.**
- Place an order for the remaining items (those in `product_ids_to_keep`).
- Be prepared to provide or fetch payment and address details if the order API requires them.
- Remain alert for edge cases, such as items going out of stock or API failures during the move/order process, and handle these gracefully.

Directive: Proceed to move each `product_id` in `product_ids_to_move` to the wish list, verify the cart, and then place the order for the remaining items.

Step 17: APIPlannerAgent

Show Prompts

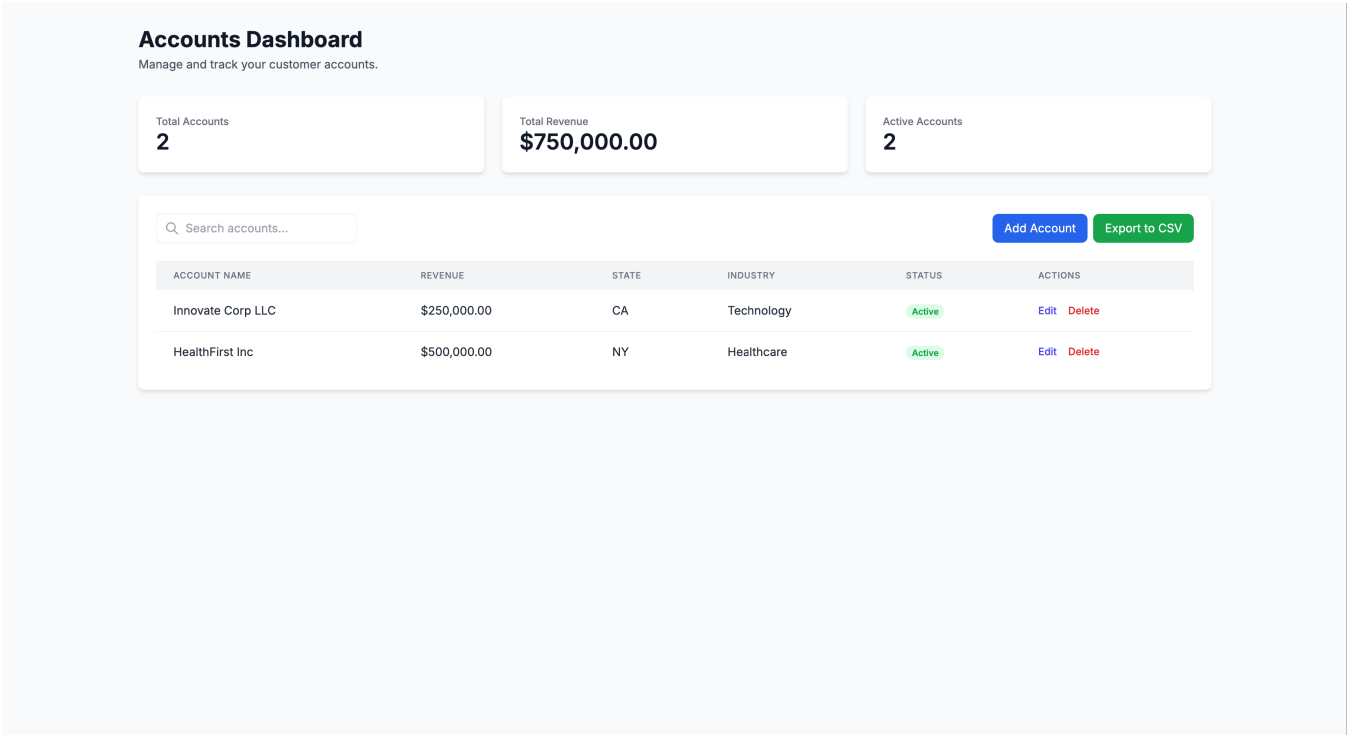
Show JSON

Interesting keys (1)

action_input_coder_agent.task_description

For each `product_id` in `cart_cheapest_selection['product_ids_to_move']`, move the product from the cart to the wish list. expected output: An array of results indicating success or failure for each `product_id` moved.

Figure 13: Trajectory showing CUGA working in a hybrid mode that involves both the browser sub-agent and the API sub-agent. The task was: 'Get the top account by revenue from digital sales, then add it to the current page.'



INTENT

get top account by revenue from digital sales then add it to current page

Show Process Summary

Filters

Devolect All

STEP TYPES

☒ TaskAnalyzerAgent

☒ TaskDecompositionAgent

☒ PlanControllerAgent

☒ APIPlannerAgent

☒ ShortlisterAgent

☒ APICodePlannerAgent

☒ CodeAgent

☒ BrowserPlannerAgent

☒ ActionAgent

☒ FinalAnswerAgent

Step 1: TaskAnalyzerAgent

Show Prompts Show JSON

Level 02 items

0:

Level 14 items

name: digital_sales

description: This Digital Sales Skills API provides sales professionals with a unified interface to access territory accounts, retrieve client information from TPP, manage job roles, and synchronize contacts between Zoominfo and Salesloft-streamlining the process of managing customer relationships and sales data across multiple platforms.

uri: https://digitalsales.19pclvtv@90u.us-east.codeengine.appdomain.cloud/openapi.json

type: api

1:

Level 14 items

name: pages.github.ibm.com

description: pages.github.ibm.com

uri:

type: web

Step 2: TaskDecompositionAgent

Show Prompts Show JSON

Level 02 items

thoughts: The intent involves retrieving data from the Digital Sales API and then utilizing the Pages application to add the retrieved information to the current page. This requires two distinct subtasks: one for data retrieval and another for updating the page.

task_decomposition: 2 items

1.

Level 13 items

task: Retrieve the top account by revenue from digital sales using the Digital Sales API.

app: digital_sales

type: api

2.

Level 13 items

task: Add the retrieved top account information to the current page on pages.github.ibm.com.

app: pages.github.ibm.com

type: web

Step 3: PlanControllerAgent

Show Prompts Show JSON

Level 07 items

thoughts: 3 items

1. The first subtask involves retrieving data from the Digital Sales API, which has not been started yet.

2. The second subtask depends on the completion of the first subtask, as it requires the retrieved data to proceed.

3. The immediate next action is to initiate the API call to retrieve the top account by revenue from digital sales.

subtasks_progress: 2 items

1. not-started

2. not-started

next_subtask: Retrieve the top account by revenue from digital sales using the Digital Sales API.

next_subtask_type: api

next_subtask_app: digital_sales

conclude_task: false

conclude_final_answer:

Step 4: APIPlannerAgent

Show Prompts Show JSON

Interesting keys 1

action_input_shortlisting_agent.task_description

Retrieve accounts with their revenue information to identify the top account by revenue.

Step 5: ShortlisterAgent

Show PromptsShow JSON

Level 02 Items

thoughts: (4 Items)

1. The user's goal is to identify the top account by revenue using the Digital Sales API. This requires retrieving account data and analyzing it to determine the account with the highest revenue.

2. The API 'digital_sales_get_my_accounts' directly provides account data, including revenue, which is essential for this task.

3. Other APIs, such as 'digital_sales_get_accounts_top', also provide account data but require additional parameters that are not specified in the user's query.

4. APIs related to job roles, contacts, and filtering contacts are not directly relevant to the task of identifying the top account by revenue.

result: (4 Items)

1.

Level 13 Items

name: digital_sales_get_my_accounts

relevance_score: 0.95

reasoning: This API directly retrieves account data, including revenue, which is necessary to identify the top account by revenue. It requires no additional parameters, making it highly relevant.

2.

Level 13 Items

name: digital_sales_get_accounts_top

relevance_score: 0.5

reasoning: This API also retrieves account data, including revenue, but requires parameters such as 'client_status', 'coverage_id', and 'campaign_name', which are not provided in the user's query. Its relevance is lower due to these additional requirements.

3.

Level 13 Items

name: digital_sales_get_job_roles

relevance_score: 0.1

reasoning: This API retrieves job titles based on a campaign name, which is unrelated to the task of identifying the top account by revenue. It is included to meet the minimum API count.

4.

Level 13 Items

name: digital_sales_get_contacts

relevance_score: 0.1

reasoning: This API retrieves contact information based on job titles and accounts, which is not directly relevant to the user's goal. It is included to meet the minimum API count.

Step 6: APIPlannerAgent

Show PromptsShow JSON

Interesting keys (1)

action_input_coder_agent_task_description

Get the top 1 account by revenue from all available accounts. expected output: A single account object with the highest revenue and its details.

Step 7: APICodePlannerAgent

Show PromptsShow JSON

```
[  
  "1. First, call the 'digital_sales.digital_sales_get_my_accounts' API to retrieve all accounts in the user's territory. Store the response in a variable called 'my_accounts_response'.",  
  "2. Extract the list of accounts from the 'my_accounts_response' under the key 'accounts'. This list will contain account objects with details such as name, state, and revenue.",  
  "3. Initialize two variables: 'top_account' to None and 'highest_revenue' to 0. These will be used to track the account with the highest revenue.",  
  "4. Loop through each account in the extracted list of accounts.",  
  "5. For each account, check its 'revenue' value. If the 'revenue' is greater than the current 'highest_revenue', update 'highest_revenue' to this account's revenue and set 'top_account' to this account object.",  
  "6. After processing all accounts, 'top_account' will contain the account with the highest revenue. If no accounts were found, prepare an error message indicating that no accounts are available.",  
  "7. Prepare the result as a JSON serializable dictionary. If an account was found, the dictionary will be {'variable_name': 'top_account_by_revenue', 'description': 'The account with the highest revenue and its details.', 'value': top_account}. Otherwise, it will be {'variable_name': 'no_accounts_found', 'description': 'No accounts were found in the user's territory.",  
  "8. Print the final result dictionary as a JSON string using 'print(json.dumps(result_dict))'."  
]
```

Step 8: CodeAgent

Show PromptsShow JSON

Interesting keys (1)

summary

Outputs: Successfully retrieved 100 accounts and identified the top account by revenue (stored in 'top_account_by_revenue').

...json

({'name': 'Summit Solutions', 'state': 'New York', 'revenue': 996142})

Step 9: APIPlannerAgent

Show PromptsShow JSON

Overall Status/Analysis: The CodeAgent successfully completed the task by retrieving 100 accounts using the most relevant API and correctly identifying the top account by revenue. The output variable, top_account_by_revenue, contains a dictionary with the expected fields (name, state, revenue), and the values are reasonable and properly formatted. There were no errors or indications of incomplete data, and the output aligns with the task requirements to retrieve the top account by revenue from digital sales.

Summary of Progress:

The ApiShortlistingAgent selected the most relevant APIs, focusing on those that provide account and revenue data.

The CodeAgent retrieved all available accounts and processed them to identify the top account by revenue, storing the result in top_account_by_revenue.

The output is clear, complete, and matches the expected format.

Skepticism/Non-Triviality Check:

While the task appears straightforward, potential hidden complexities could include incomplete account data, inconsistencies in the revenue field, or missing accounts due to API limitations. However, the agent reports retrieving 100 accounts, suggesting a comprehensive dataset.

There was no filtering by territory, client status, or other constraints, which matches the user's broad request.

Edge cases such as ties in revenue or missing revenue fields do not appear to have affected the result, but could be considered in future iterations if needed.

Strategic Recommendation:

The task is complete and the output meets the requirements. No further action is needed unless you request additional details, filtering, or validation of the dataset.

Step 10: APIPlannerAgent

Show Prompts Show JSON

Interesting keys (1)

action_input_conclude_task_final_response

The top account by revenue from digital sales is: {'name': 'Summit Solutions', 'state': 'New York', 'revenue': 996142}. This result was obtained by retrieving all available accounts and selecting the one with the highest revenue using the Digital Sales API.

Step 11: PlanControllerAgent

Show Prompts Show JSON

Level 07 Items

thoughts: (4 Items)

1. The first subtask 'Retrieve the top account by revenue from digital sales' is completed, and the result is stored in the variable `top_account_by_revenue`.

2. The second subtask 'Add the retrieved top account information to the current page' is not-started.

3. The next logical action is to proceed with the second subtask, utilizing the data stored in `top_account_by_revenue`.

4. The planner needs to interact with the current webpage to add the account details.

subtasks_progress: (2 Items)

1. completed

2. not-started

next_subtask: Using the details from `top_account_by_revenue`, add the account information (name, state, and revenue) to the current page on `pages.github.ibm.com`.

next_subtask_type: web

next_subtask_app:

conclude_task: false

conclude_final_answer:

Step 12: BrowserPlannerAgent

Show Prompts Show JSON

RootWebArea 'Accounts Management', focused, url='https://pages.github.ibm.com/research-rpa/cuga/sales_v1.html'

[11] banner ''

[12] heading 'Accounts Dashboard'

[13] paragraph ''

StaticText 'Manage and track your customer accounts.'

[16] heading 'Total Accounts'

[17] paragraph ''

StaticText

Level 03 Items

thoughts: (3 Items)

1. The original task is to add the account from `'top_account_by_revenue'` to the current page.

2. The 'Add Account' button is visible and likely starts the process to add a new account.

3. The next step is to click the 'Add Account' button to begin entering the new account details.

next_agent: ActionAgent

instruction: Click the 'Add Account' button.

🔦

Step 13: ActionAgent

Show Prompts ⌵

Show JSON ⌵

click(bid=30)

🔦

Step 14: BrowserPlannerAgent

Show Prompts ⌵

Show JSON ⌵

RootWebArea 'Accounts Management', focused, url='https://pages.github.ibm.com/research-rpa/cuga/sales_v1.html'
[11] banner ''
[12] heading 'Accounts Dashboard'
[13] paragraph ''
StaticText 'Manage and track your customer accounts.'
[16] heading 'Total Accounts'
[17] paragraph ''
StaticTe

▼ Level 0 3 items

🧠 thoughts: (3 items)

1. The 'Add Account' modal is open with fields for Account Name, Revenue, State, Industry, and Status.

2. The original task requires entering the details from 'top_account_by_revenue', which are: name: 'Summit Solutions', state: 'New York', revenue: 996142.

3. The first logical step is to type the account name 'Summit Solutions' into the 'Account Name' textbox.

next_agent: ActionAgent

instruction: Type 'Summit Solutions' into the 'Account Name' textbox.

🔦

Step 15: ActionAgent

Show Prompts ⌵

Show JSON ⌵

type(bid=70, value=Summit Solutions, press_enter=False)

🔦

Step 16: BrowserPlannerAgent

Show Prompts ⌵

Show JSON ⌵

RootWebArea 'Accounts Management', focused, url='https://pages.github.ibm.com/research-rpa/cuga/sales_v1.html'
[11] banner ''
[12] heading 'Accounts Dashboard'
[13] paragraph ''
StaticText 'Manage and track your customer accounts.'
[16] heading 'Total Accounts'
[17] paragraph ''
StaticTe

▼ Level 0 3 items

🧠 thoughts: (4 items)

1. The 'Account Name' field has been filled with 'Summit Solutions'.

2. The next required field is 'Revenue', which should be set to 996142 as per the variable 'top_account_by_revenue'.

3. The 'Revenue' field is a spinbutton and currently empty.

4. The logical next step is to type '996142' into the 'Revenue' spinbutton.

next_agent: ActionAgent

instruction: Type '996142' into the 'Revenue' spinbutton.

🔦

Step 17: ActionAgent

Show Prompts ⌵

Show JSON ⌵

type(bid=71, value=996142, press_enter=False)

🔦

Step 18: BrowserPlannerAgent

Show Prompts ⌵

Show JSON ⌵

RootWebArea 'Accounts Management', focused, url='https://pages.github.ibm.com/research-rpa/cuga/sales_v1.html'
[11] banner ''
[12] heading 'Accounts Dashboard'
[13] paragraph ''
StaticText 'Manage and track your customer accounts.'
[16] heading 'Total Accounts'
[17] paragraph ''
StaticTe

▼ Level 0 3 items

🧠 thoughts: (4 items)

1. The 'Account Name' and 'Revenue' fields have been filled with 'Summit Solutions' and '996142' respectively.

2. The next required field is 'State', which should be filled with 'New York' as per the variable 'top_account_by_revenue'.

3. The 'State' textbox is currently empty and required.

4. The logical next step is to type 'New York' into the 'State' textbox.

next_agent: ActionAgent

instruction: Type 'New York' into the 'State' textbox.

🔗

Step 19: ActionAgent

Show Prompts 📄

Show JSON 📄

```
type(bid=72, value=New York, press_enter=False)
```

🔗

Step 20: BrowserPlannerAgent

Show Prompts 📄

Show JSON 📄

```
RootWebArea 'Accounts Management', focused, url='https://pages.github.ibm.com/research-rpa/cuga/sales_v1.html'  
[11] banner ''  
[12] heading 'Accounts Dashboard'  
[13] paragraph ''  
      StaticText 'Manage and track your customer accounts.'  
[16] heading 'Total Accounts'  
[17] paragraph ''  
      StaticTe
```

📌 Level 0 3 items

💡 thoughts: (4 items)

1. The 'Account Name', 'Revenue', and 'State' fields have been filled with the values from 'top_account_by_revenue'.

2. The next required field is 'Industry', which is currently empty and needs to be filled to proceed with adding the account.

3. According to 'top_account_by_revenue', the industry value is missing, so I need to check if it should be left blank or if a default value should be entered.

4. Since the original task only specifies name, state, and revenue, and the industry is not provided, I will leave it blank and proceed to save the account.

next_agent: ActionAgent

instruction: Click the 'Save' button to submit the new account information.

🔗

Step 21: ActionAgent

Show Prompts 📄

Show JSON 📄

```
click(bid=79)
```

🔗

Step 22: BrowserPlannerAgent

Show Prompts 📄

Show JSON 📄

```
RootWebArea 'Accounts Management', focused, url='https://pages.github.ibm.com/research-rpa/cuga/sales_v1.html'  
alert 'Please fill out this field.', live='assertive', atomic, relevant='additions'  
[11] banner ''  
[12] heading 'Accounts Dashboard'  
[13] paragraph ''  
      StaticText 'Manage and track
```

📌 Level 0 3 items

💡 thoughts: (4 items)

1. The previous step attempted to save the new account, but the form was not submitted due to a required 'Industry' field being empty.

2. The alert 'Please fill out this field.' indicates that the 'Industry' textbox must be filled before the form can be submitted.

3. Since 'top_account_by_revenue' does not provide an industry, I should enter a default value such as 'General' to proceed.

4. After entering a value for 'Industry', I can attempt to save the account again.

next_agent: ActionAgent

instruction: Type 'General' into the 'Industry' textbox.

🔗

Step 23: ActionAgent

Show Prompts 📄

Show JSON 📄

```
type(bid=73, value=General, press_enter=False)
```

🔗

Step 24: BrowserPlannerAgent

Show Prompts 📄

Show JSON 📄

```
RootWebArea 'Accounts Management', focused, url='https://pages.github.ibm.com/research-rpa/cuga/sales_v1.html'  
[11] banner ''  
[12] heading 'Accounts Dashboard'  
[13] paragraph ''  
      StaticText 'Manage and track your customer accounts.'  
[16] heading 'Total Accounts'  
[17] paragraph ''  
      StaticTe
```

📌 Level 0 3 items

💡 thoughts: (2 items)

1. The 'Industry' field has now been filled with 'General', which should satisfy the form's requirements.

2. The next logical step is to click the 'Save' button to submit the new account information and complete the account addition process.

next_agent: ActionAgent

instruction: Click the 'Save' button to submit the new account information.

Step 25: ActionAgent

Show Prompts Show JSON

click(bid=79)

Step 26: BrowserPlannerAgent

Show Prompts Show JSON

RootWebArea 'Accounts Management', focused, url='https://pages.github.ibm.com/research-rpa/cuga/sales_v1.html'
[11] banner ''
[12] heading 'Accounts Dashboard'
[13] paragraph ''
StaticText 'Manage and track your customer accounts.'
[16] heading 'Total Accounts'
[17] paragraph ''
StaticText

Level 0 3 items

thoughts: (2 items)

1. The account 'Summit Solutions' with state 'New York' and revenue '996142' has been successfully added to the accounts table, as verified by the presence of this entry in the table on the current page.

2. The original task was to add the account information from 'top_account_by_revenue' to the page, which is now complete.

next_agent: ConcludeTaskAgent

instruction: The account 'Summit Solutions' with state 'New York' and revenue '996142' has been successfully added to the accounts table. The original task is complete.

Step 27: PlanControllerAgent

Show Prompts Show JSON

Level 0 7 items

thoughts: (3 items)

1. The first subtask 'Retrieve the top account by revenue from digital sales using the Digital Sales API' is completed, as the variable 'top_account_by_revenue' contains the required data.

2. The second subtask 'Add the retrieved top account information to the current page on pages.github.ibm.com' is also completed, as the account 'Summit Solutions' has been successfully added to the page.

3. All subtasks are completed, and the intent has been fulfilled.

subtasks_progress: (2 items)

1. completed

2. completed

next_subtask:

next_subtask_type: web

next_subtask_app:

conclude_task: true

conclude_final_answer: The task has been successfully completed. The top account by revenue, 'Summit Solutions', has been retrieved and added to the current page.

Step 28: FinalAnswerAgent

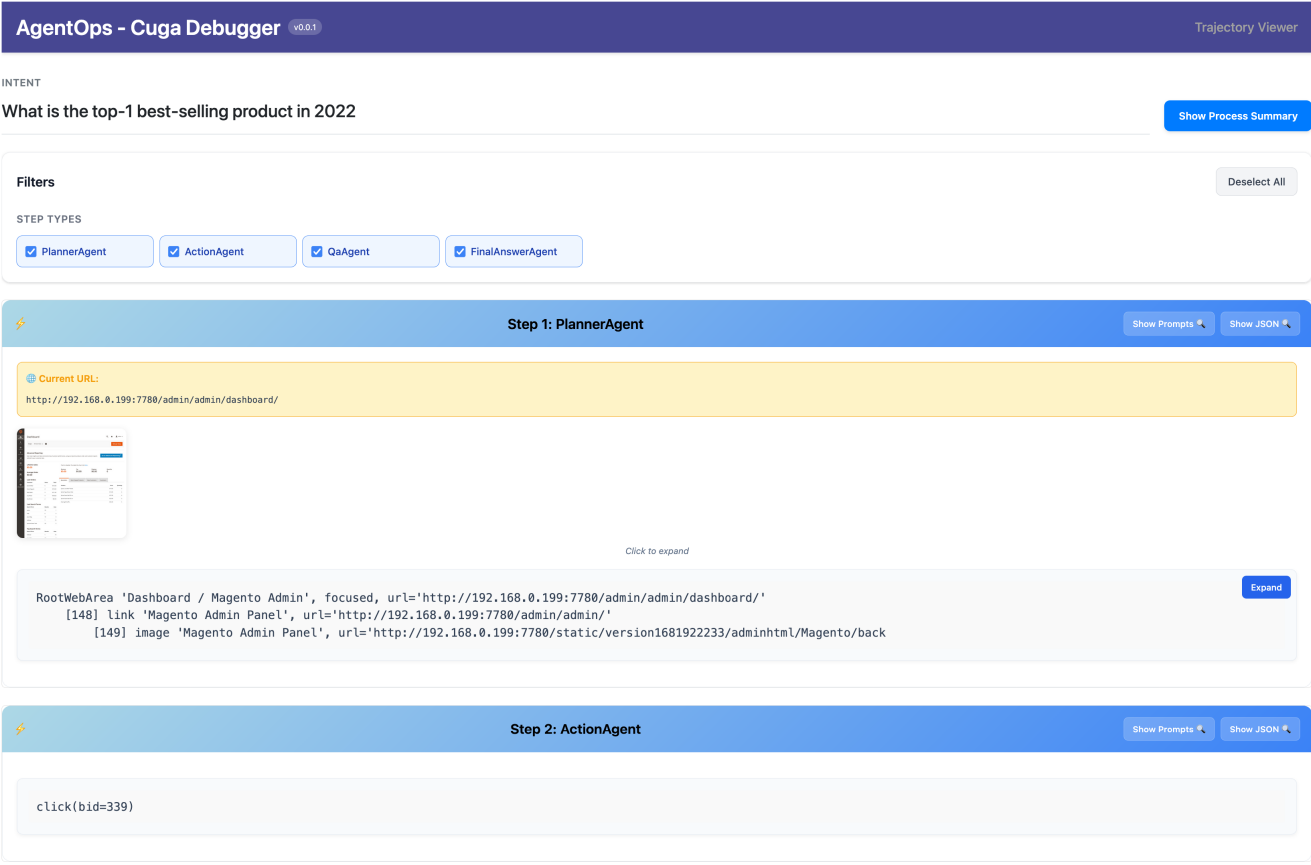
Show Prompts Show JSON

Level 0 2 items

thoughts: (empty array)

final_answer: The top account by revenue, Summit Solutions (New York, \$996,142), has been successfully retrieved and added to the current page.

Figure 14: The trajectory shows CUGA succeeded a task in the WebArena benchmark using the browser sub-agent.





Step 3: PlannerAgent

Show Prompts

Show JSON

Current URL:

http://192.168.0.199:7780/admin/admin/dashboard/



Click to expand

RootWebArea 'Dashboard / Magento Admin', focused, url='http://192.168.0.199:7780/admin/admin/dashboard/'
[148] link 'Magento Admin Panel', url='http://192.168.0.199:7780/admin/admin/'
[149] image 'Magento Admin Panel', url='http://192.168.0.199:7780/static/version1681922233/adminhtml/Magento/back

Expand



Step 4: ActionAgent

Show Prompts

Show JSON

click(bid=431)



Step 5: PlannerAgent

Show Prompts

Show JSON

Current URL:

http://192.168.0.199:7780/admin/reports/report_sales/bestsellers/



Click to expand

RootWebArea 'Bestsellers Report / Products / Reports / Magento Admin', focused, url='http://192.168.0.199:7780/admin/reports/report_sales/bestsellers/'
[149] link 'Magento Admin Panel', url='http://192.168.0.199:7780/admin/admin/'
[150] image 'Magento Admin Panel', url='http://192.168.0.199:7780/

Expand



Step 6: ActionAgent

Show Prompts

Show JSON

```
select_option(bid=741, options=Year)
```



Step 7: PlannerAgent

Show Prompts

Show JSON

 Current URL:

http://192.168.0.199:7780/admin/reports/report_sales/bestsellers/



Click to expand

RootWebArea 'Bestsellers Report / Products / Reports / Magento Admin', focused, url='http://192.168.0.199:7780/admin/reports/report_sales/bestsellers/'

[149] link 'Magento Admin Panel', url='http://192.168.0.199:7780/admin/admin/'

[150] image 'Magento Admin Panel', url='http://192.168.0.199:7780/'

Expand



Step 8: ActionAgent

Show Prompts

Show JSON

```
type(bid=749, value=01/01/2022, press_enter=False)
```



Step 9: PlannerAgent

Show Prompts

Show JSON

 Current URL:

http://192.168.0.199:7780/admin/reports/report_sales/bestsellers/



Click to expand

RootWebArea 'Bestsellers Report / Products / Reports / Magento Admin', focused, url='http://192.168.0.199:7780/admin/reports/report_sales/bestsellers/'

[149] link 'Magento Admin Panel', url='http://192.168.0.199:7780/admin/admin/'

[150] image 'Magento Admin Panel', url='http://192.168.0.199:7780/'

Expand



Step 10: ActionAgent

[Show Prompts](#)[Show JSON](#)

```
type(bid=756, value=12/31/2022, press_enter=False)
```



Step 11: PlannerAgent

[Show Prompts](#)[Show JSON](#)

Current URL:

http://192.168.0.199:7780/admin/reports/report_sales/bestsellers/



[Click to expand](#)

RootWebArea 'Bestsellers Report / Products / Reports / Magento Admin', focused, url='http://192.168.0.199:7780/admin/reports/report_sales/bestsellers/'
[149] link 'Magento Admin Panel', url='http://192.168.0.199:7780/admin/admin/'
[150] image 'Magento Admin Panel', url='http://192.168.0.199:7780/

[Expand](#)

Step 12: ActionAgent

[Show Prompts](#)[Show JSON](#)

```
click(bid=714)
```



Step 13: PlannerAgent

[Show Prompts](#)[Show JSON](#)

Current URL:

http://192.168.0.199:7780/admin/reports/report_sales/bestsellers/filter/cGvYw9kX3R5cGU9eWVhcnZmcm9tPTAxJTJGMDEUMkYyMDIyJnRvPTEyJTJGMzEUMkYyMDIyJnNob3dfZW1wdHlfcm93cz0w/



[Click to expand](#)

RootWebArea 'Bestsellers Report / Products / Reports / Magento Admin', focused, url='http://192.168.0.199:7780/admin/reports/report_sales/bestsellers/filter/cGvYw9kX3R5cGU9eWVhcnZmcm9tPTAxJTJGMDEUMkYyMDIyJnRvPTEyJTJGMzEUMkYyMDIyJnNob3dfZW1wdHlfcm93cz0w/'
[149] link 'Magento Admin Panel', url='http

[Expand](#)

Step 14: QaAgent

answer: The top-1 best-selling product in 2022 is the "Quest Lumaflex™ Band" with an order quantity of 5.

Step 15: PlannerAgent

 Current URL:

http://192.168.0.199:7780/admin/reports/report_sales/bestsellers/filter/cGVyYW9kX3R5cGU9eWVhczZmcm9tPTAxJTJGMDElMkYyMDIyJnRvPTEyJTJGMzElMkYyMDIyJnNob3dfZW1wdHlfcmc93cz0w/



RootWebArea 'Bestsellers Report / Products / Reports / Magento Admin', focused, url='http://192.168.0.199:7780/admin/reports/report_sales/bestsellers/filter/cGvYyab9KX3R5cGJ9eWVhczIzmc9rPTAxTjGMDElMkYyMDIyJnRvPTEyTjJGMzE1MkYyMDIyJnNo3dfZWlwdHlfc9m3cz0w/',

Expand

Step 16: FinalAnswerAgent

Show Prompts 🔍

final_answer: Quest Lumaflex™ Band